SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION, AFFILIATED TO ANNA UNIVERSITY)

RAJIV GANDHI SALAI (OMR), KALAVAKKAM – 630110.

LABORATORY RECORD

NAME : SHARRUK S

REG. NO : 3122247001061

DEPT : 5YI MTECH CSE **SEM** : V

SRI SIVASUBRAMANIYA NADAR COLLEGE OF ENGINEERING

(AN AUTONOMOUS INSTITUTION, AFFILIATED TO ANNA UNIVERSITY)

RAJIV GANDHI SALAI (OMR), KALAVAKKAM – 630110.

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of the practical work done in the

ICS1561 – ETHICAL HACKING Laboratory by

Name : SHARRUK S

Register Number : 3122247001061

Semester : V

Branch : 5YI MTECH CSE

During the Academic Year : 2026 - 2027

Faculty **Head of the Department**

Submitted for the End Semester Practical Examination held at SSNCE on

_____.

Internal Examiner External Examiner

INDEX

NAME: SHARRUK S REG. NO: 3122247001061

SEM: V

S. No	Date	Experiment	Page No	Signature
1	24/06/2026	Install Kali or Backtrack Linux / Metasploitable/ Windows XP	4	
2	01/07/2026 & 15/07/2026	Practice the basics of reconnaissance	21	
3	22/07/2026	Using FOCA / SearchDiggity tools, extract metadata and expanding the target list	39	
4	29/07/2026	Aggregates information from public databases using online free tools like Paterva's Maltego.	46	
5	12/08/2026	View and capture network traffic using Wireshark.	62	
6	09/09/2026	Scan the target using tools like Nessus.	68	
7	16/09/2026	Information gathering using tools like Robtex.	79	
8	23/09/2026	Automate dig for vulnerabilities and match exploits using Armitage	84	

Ex. No. 1 **Date:** 24/06/2026

Install Kali or Backtrack Linux / Metasploitable/ Windows XP

Objective:

To build an isolated, authorised ethical-hacking laboratory by installing Kali Linux (attacker), Metasploitable 2 (vulnerable target) and Windows XP in Oracle VirtualBox and verifying connectivity between them; and, within this lab, to carry out the first monitoring and discovery tasks – endpoint logging with Sysmon, OSINT email/host enumeration with theHarvester, and ARP-based local network discovery with Netdiscover.

Tools:

Oracle VirtualBox, Kali Linux (x64), Metasploitable 2, Windows XP Professional SP3, Sysmon (Sysinternals) with Windows Event Viewer, theHarvester, Netdiscover.

Methodology:

The experiment is carried out in four parts (1.1 to 1.4). All active steps are performed only on the isolated virtual lab network.

1.1 Kali Linux + Metasploitable Lab Setup

Principle: A penetration-testing lab keeps deliberately vulnerable machines away from production networks. Kali Linux is the security-testing workstation, Metasploitable 2 provides intentionally vulnerable services, and virtualization lets both share a Host-Only/internal network without exposing the target to the Internet.

A. Install VirtualBox

1. Download the Windows-hosts installer from virtualbox.org and run it with the default options.
2. Click Yes on the network-disconnection warning, click Install, allow administrator permission and click Finish.
3. Open VirtualBox and confirm that the VirtualBox Manager window appears.

B. Install Kali Linux

1. Download the Installer image (x86 64-bit, latest stable) from kali.org.
2. Click New; Name: kali-linux, Type: Linux, Version: Debian (64-bit); Base Memory 2048 MB; create a VDI virtual hard disk of 20 GB.
3. Attach the Kali ISO, start the VM and choose Graphical Install.
4. Select language English, location India, keyboard American English; hostname kali; create the user account and password.
5. Partitioning: Guided – use entire disk, All files in one partition, Finish partitioning and write changes to disk = Yes.
6. Enable the network package manager (Yes), install GRUB on /dev/sda, click Continue to reboot and log in to the Kali desktop.

C. Install Metasploitable 2

1. Download Metasploitable2-Linux.zip (Rapid7) and extract it to obtain Metasploitable.vmdk.
2. Click New; Name: Metasploitable2, Type: Linux, Version: Ubuntu; choose Use an Existing Virtual Hard Disk File and select Metasploitable.vmdk.
3. Settings > Network: Adapter 1 = Host-Only Adapter; optional Adapter 2 = NAT.
4. Start the VM and log in with the default credentials msfadmin / msfadmin; run ip a to note its IP address.

D. Install Windows XP

1. Download the Windows XP Professional SP3 (x86) ISO; click New; Name: Windows XP, Type: Microsoft Windows, Version: Windows XP (32-bit); create a VDI disk.
2. Settings > Storage: attach the XP ISO to the empty optical drive and start the VM.
3. In text-mode setup press Enter, F8 to accept the licence, select the unpartitioned space and format it as NTFS (Quick).
4. After the automatic reboot complete the graphical setup (region, name, product key, computer name/password, date and time, typical network settings).
5. Remove the ISO (Devices > Optical Drives > Remove disk), restart, open cmd with Windows+R and run ipconfig to note the IP address.

E. Verify connectivity

1. Identify the interface address and routes on Kali, then ping the target from Kali. Replies confirm IP connectivity.

```
ip a
ip route
ping <target-ip>
ifconfig      # available on Metasploitable
```

Observation: Metasploitable eth0 obtained 10.0.2.15/24 and four ICMP replies were received from Kali. Kali = attacker workstation; Metasploitable = training target (its login banner warns never to expose it to an untrusted network), so only the isolated authorised network is used.

1.2 Endpoint Monitoring with Sysmon

Principle: Sysmon (System Monitor) is a free Microsoft Sysinternals Windows service and driver that records detailed endpoint activity – process creation, parent–child relationships, command lines, network connections, file and registry activity – into the Windows Event Log. It is a monitoring source, not a verdict engine, so events must be interpreted in context.

1. Download Sysmon from the Sysinternals page (Microsoft Learn) and extract the ZIP file.
2. Download a Sysmon configuration file (SwiftOnSecurity sysmon-config, sysmonconfig-export.xml) into the same folder.
3. Open PowerShell/Command Prompt as Administrator and change to the Sysmon folder.
4. Install Sysmon with the default configuration, or with the downloaded configuration file; the Sysmon64 service and SysmonDrv driver start.

5. Open Event Viewer > Applications and Services Logs > Microsoft > Windows > Sysmon > Operational, launch a few normal applications, refresh, and inspect process name, parent process, user, command line and timestamp.

```
Sysmon64.exe -accepteula -i
Sysmon64.exe -accepteula -i sysmonconfig-export.xml
Sysmon64.exe -c                    # view current configuration
Sysmon64.exe -c sysmonconfig.xml     # update rules without reinstalling
Sysmon64.exe -u force              # uninstall without confirmation
Sysmon64.exe -h                    # help
```

Observation: Event ID 1 = process creation, ID 3 = network connection, ID 11 = file creation, ID 13 = registry value set, ID 22 = DNS query. Sysmon records useful endpoint events that can be reviewed in Event Viewer.

1.3 Email Enumeration with theHarvester

Principle: theHarvester is an OSINT reconnaissance utility that aggregates data from public sources (search engines, certificate transparency, DNS datasets, etc.) to list email addresses, hostnames, subdomains and IP addresses of a target domain. Results can be stale, so important findings must be verified before reporting.

1. Update the package list and install theHarvester on Kali; verify with the version and help options.
2. Choose the instructor-provided or example domain (microsoft.com is used here as a public example).
3. Run the query with -d (target domain), -b (data source), -l (result limit) and -q (quiet mode).
4. Review the discovered emails, IP addresses, hosts and subdomains and record the relevant findings.

```
sudo apt update
sudo apt install theharvester
theHarvester --version
theHarvester -h
theHarvester -d microsoft.com -b all -l 300 -q
```

Observation: Several sources were skipped because no API key was configured, but the remaining sources returned 119 IP addresses plus a large list of hosts/subdomains. Email naming patterns can reveal organisational structure; the data is public, but scope and privacy still matter.

1.4 Local Network Discovery with Netdiscover

Principle: Netdiscover is an active/passive ARP reconnaissance tool. ARP resolves an IPv4 address to a MAC address, so every host that answers can be listed with its IP, MAC address and vendor. It works on the same broadcast domain (LAN) and does not cross routers.

1. Install Netdiscover and display its usage; confirm the interface and subnet with ip a.
2. To scan the LAN from the VM, open VirtualBox Settings > Network and change Adapter 1 from NAT to Bridged Adapter.
3. Scan with the default interface, then restrict the scan to a known range, a specific interface, or use passive mode (listens for ARP traffic without sending requests).
4. Record the IP address, MAC address and vendor of each discovered device.

```
sudo apt install netdiscover
```

```
netdiscover -h
sudo netdiscover
sudo netdiscover -r 192.168.1.0/24
sudo netdiscover -i eth0
sudo netdiscover -p
```

Observation: Netdiscover listed 20+ hosts with IP, MAC address and vendor (e.g. Intel, Apple, Tejas Networks); passive mode found fewer hosts (13) because it only hears devices that happen to send ARP traffic.

Screenshots:

1.1 Kali Linux + Metasploitable Lab Setup

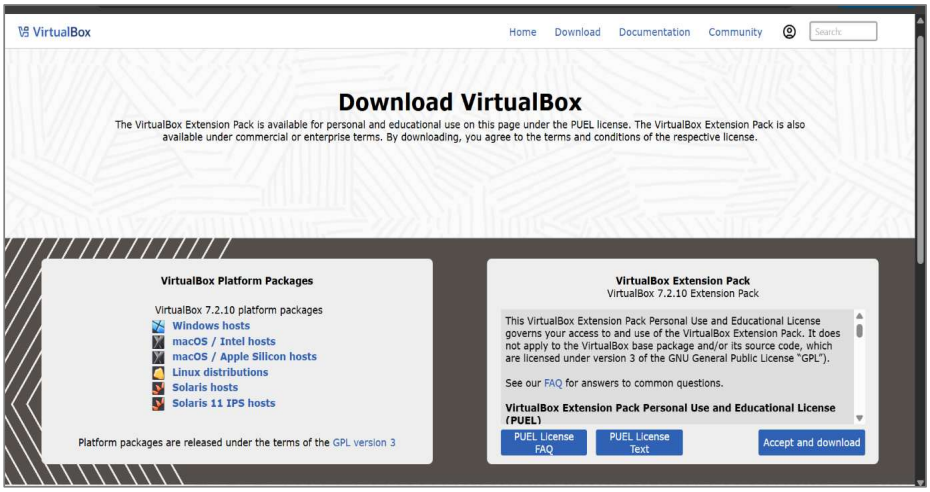


Fig. 1.1 VirtualBox download page



Fig. 1.2 VirtualBox installation completed

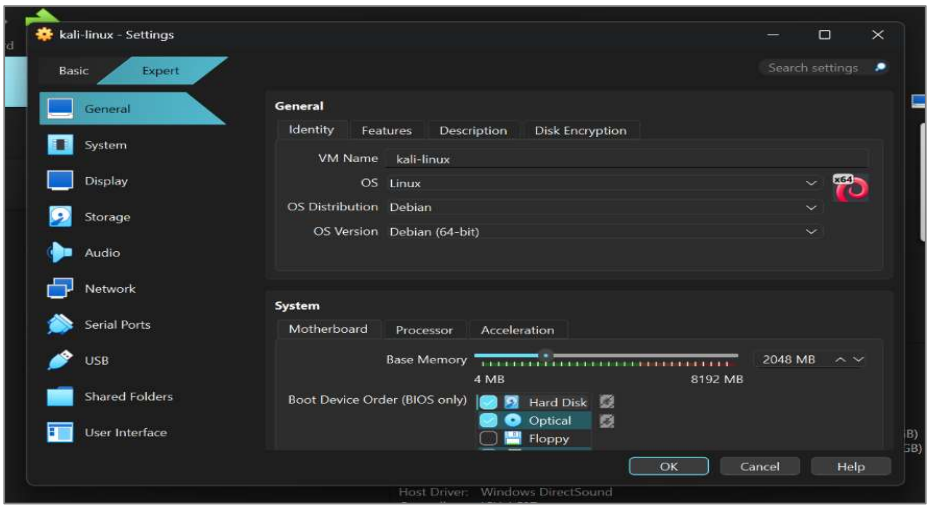


Fig. 1.3 Kali VM settings (Debian 64-bit, 2048 MB RAM)

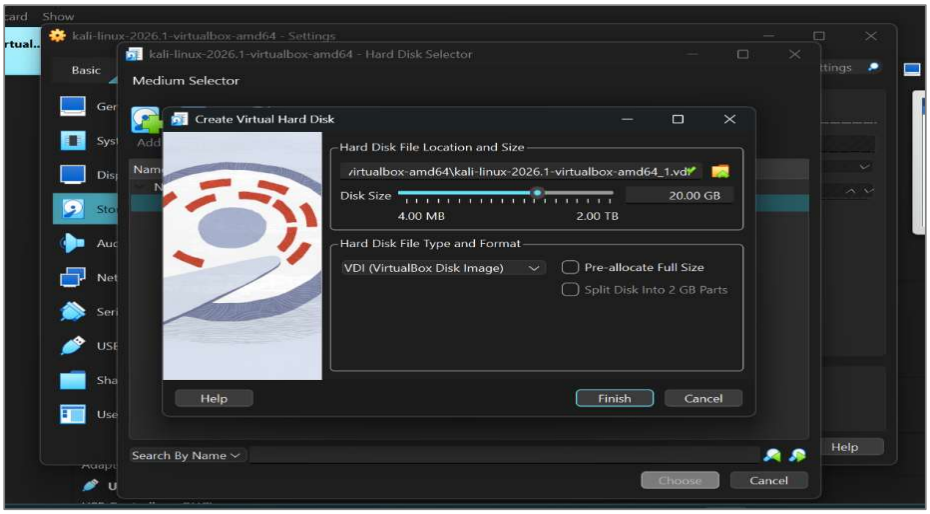


Fig. 1.4 Creating the 20 GB VDI virtual disk



Fig. 1.5 Kali boot menu – Graphical Install

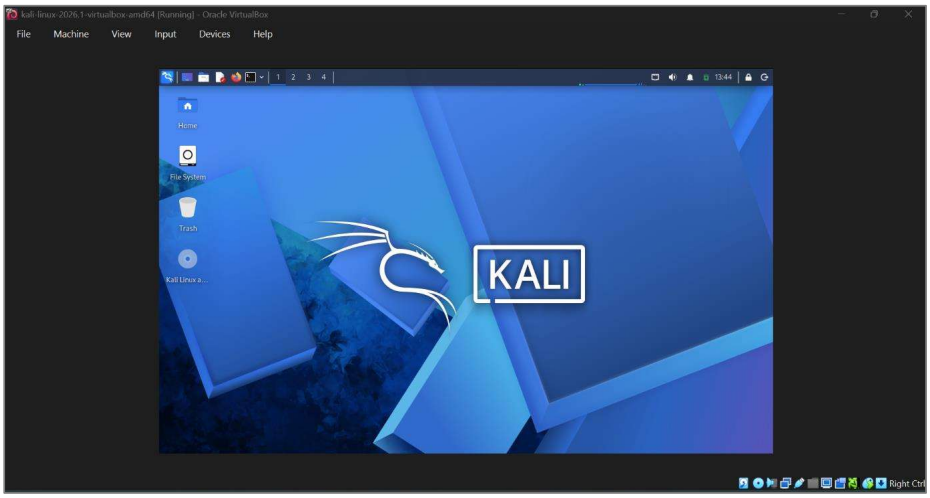


Fig. 1.6 Kali Linux desktop after first login

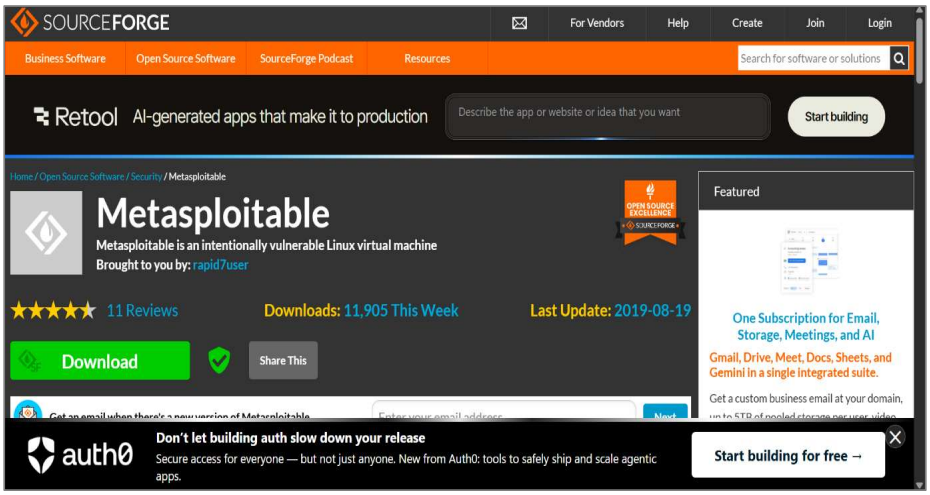


Fig. 1.7 Metasploitable 2 download page

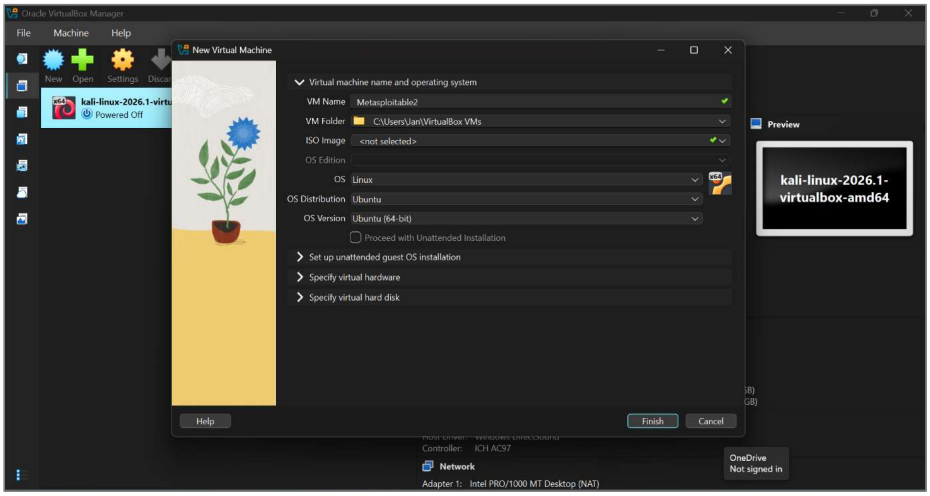


Fig. 1.8 Creating the Metasploitable2 VM with the existing .vmdk disk

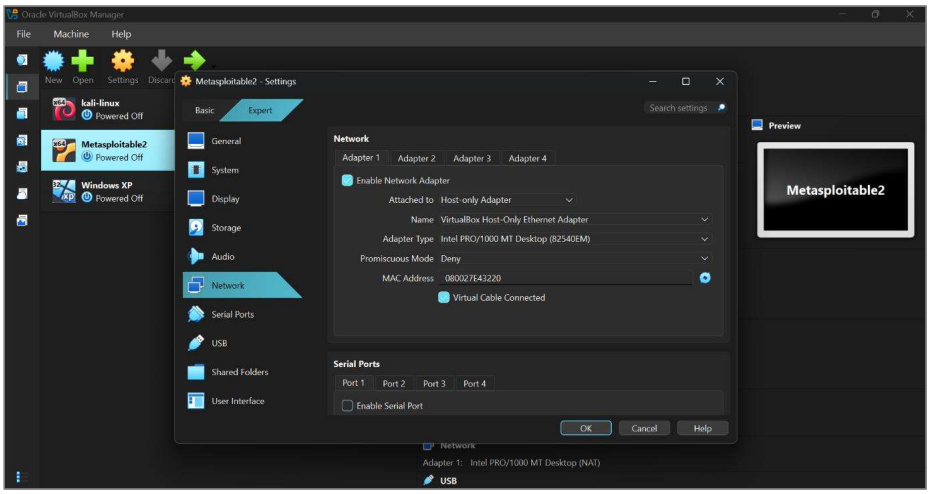


Fig. 1.9 Metasploitable2 Adapter 1 – Host-Only Adapter

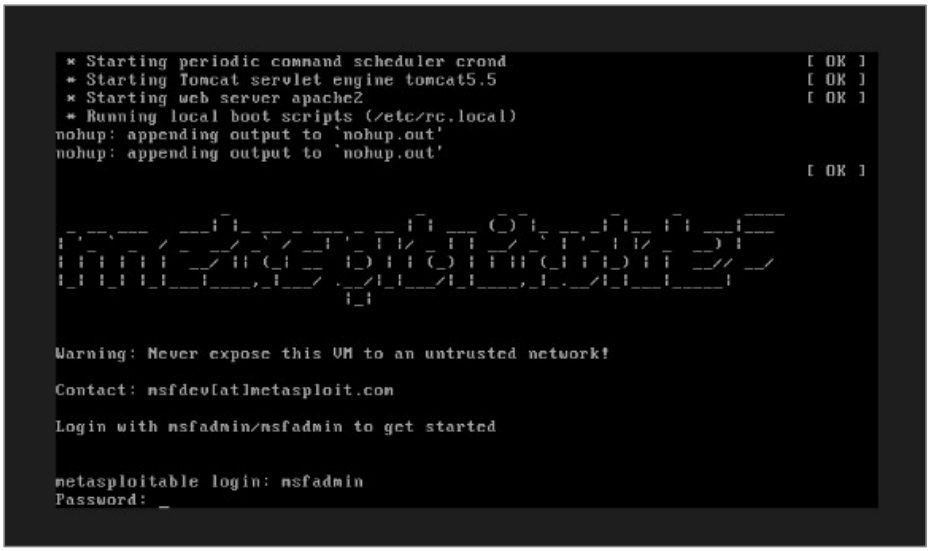


Fig. 1.10 Metasploitable login screen (msfadmin)



Fig. 1.11 ip a on Metasploitable – eth0 10.0.2.15/24

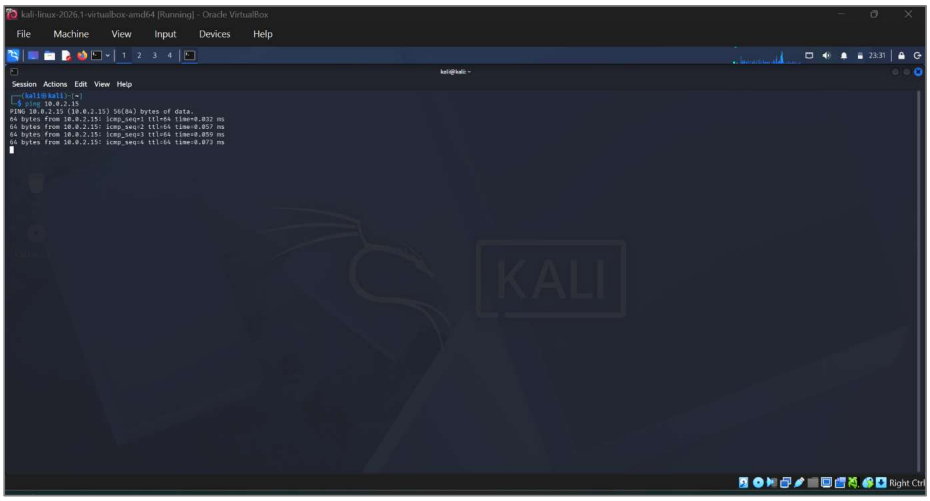


Fig. 1.12 Ping from Kali to Metasploitable – successful replies



Fig. 1.13 Windows XP setup – partition formatted as NTFS



Fig. 1.14 Windows XP desktop after installation

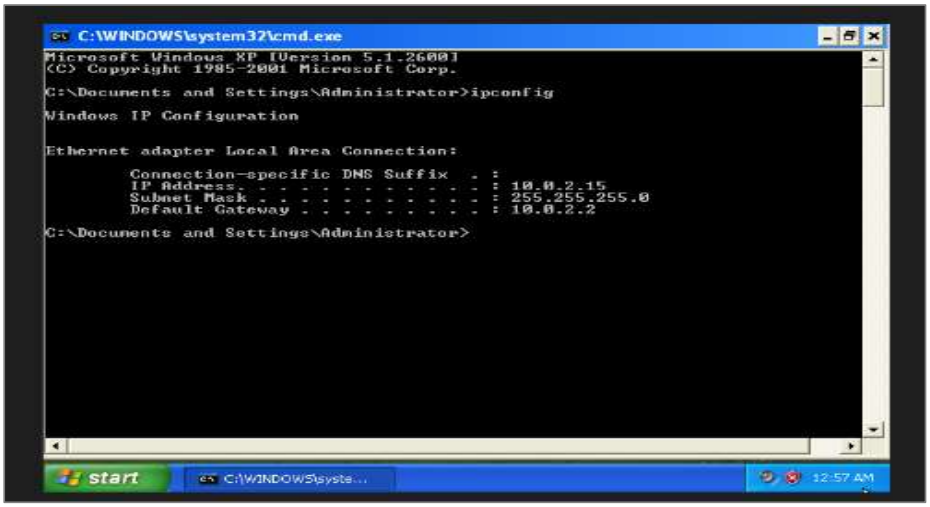


Fig. 1.15 ipconfig on Windows XP

1.2 Endpoint Monitoring with Sysmon

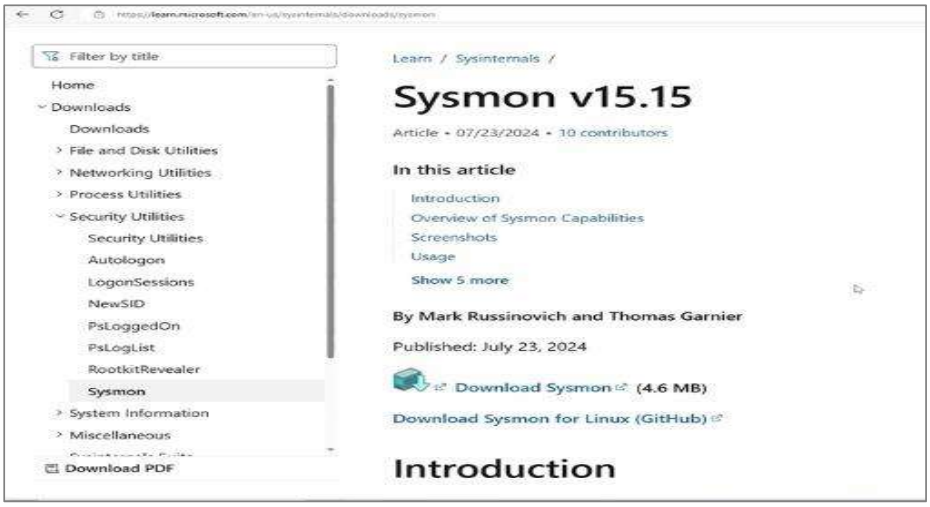


Fig. 1.16 Sysmon download page (Sysinternals)

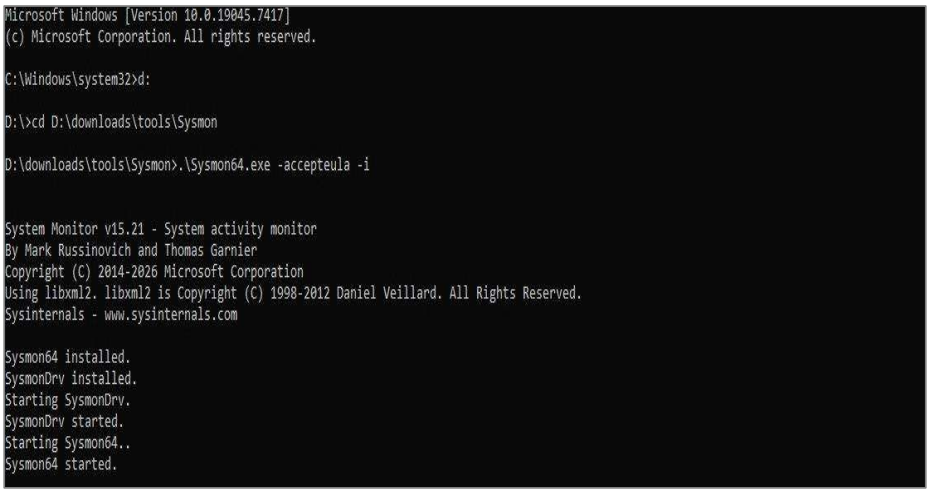


Fig. 1.17 Sysmon installed with default configuration

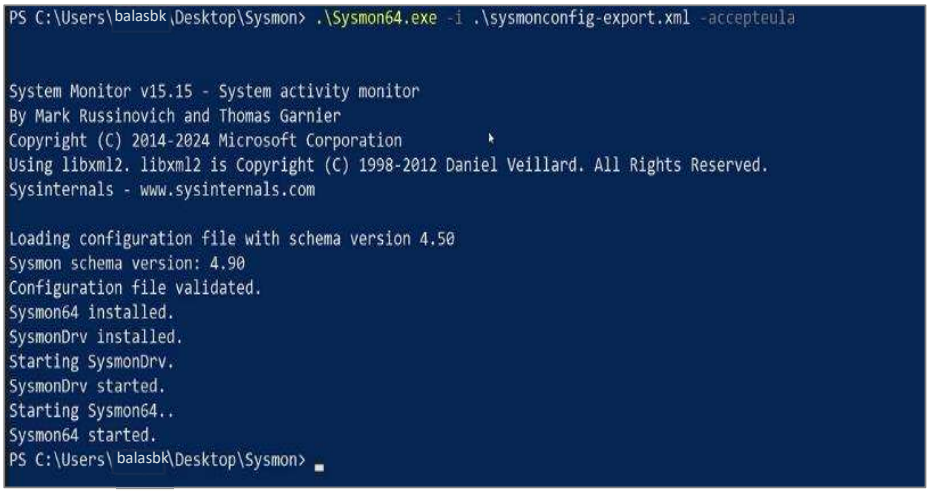


Fig. 1.18 Sysmon installed with sysmonconfig-export.xml

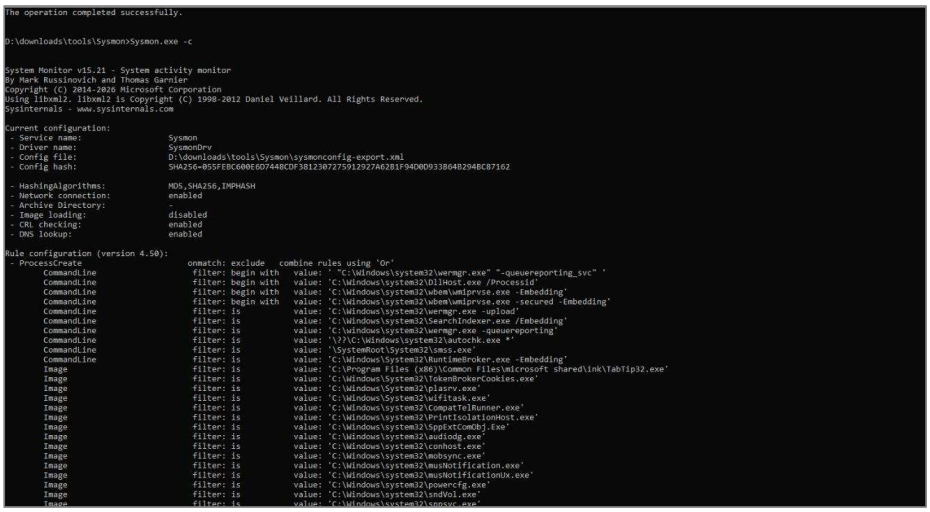


Fig. 1.19 Current Sysmon configuration (Sysmon64.exe -c)

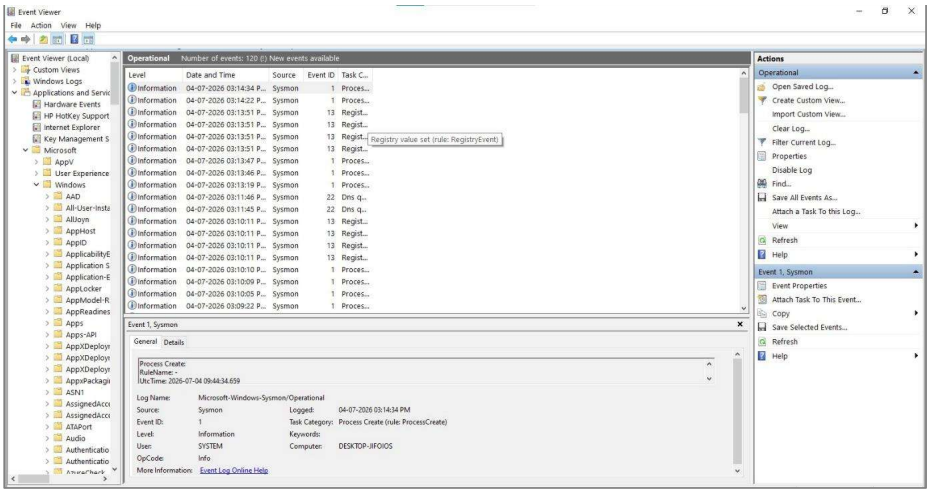


Fig. 1.20 Sysmon events in Event Viewer (Event ID 1 – Process Create)

1.3 Email Enumeration with theHarvester



Fig. 1.21 theHarvester installation verified



Fig. 1.22 theHarvester run: -d microsoft.com -b all -l 300 -q



Fig. 1.23 IP addresses found

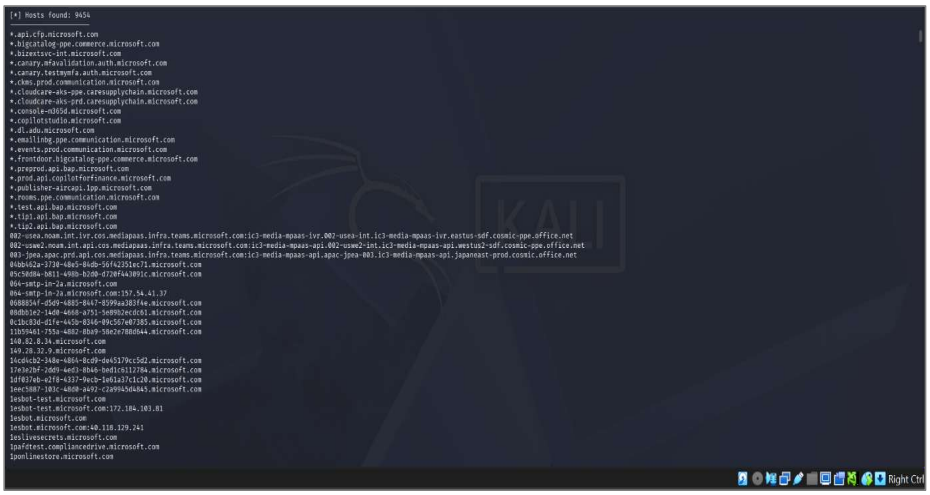


Fig. 1.24 Hosts / subdomains found

1.4 Local Network Discovery with Netdiscover



Fig. 1.25 Netdiscover usage / options

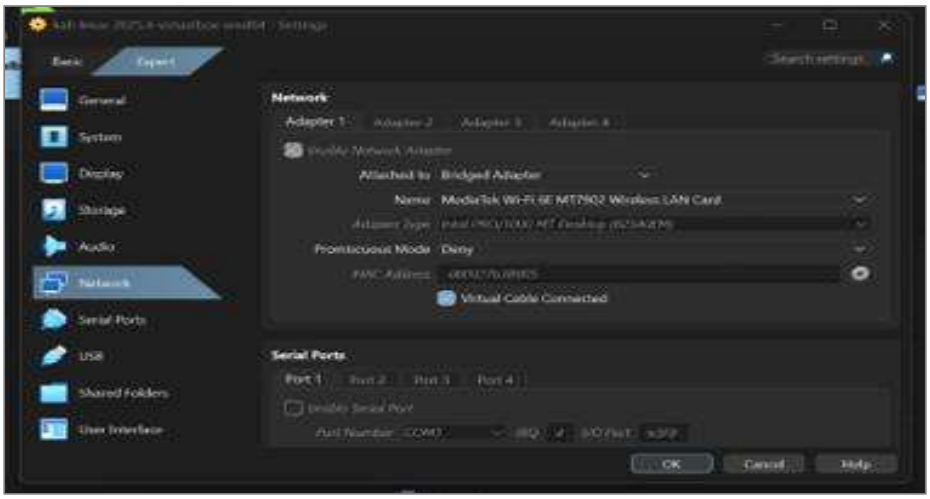


Fig. 1.26 VirtualBox network adapter changed to Bridged Adapter



Fig. 1.27 netdiscover -r 192.168.1.0/24



Fig. 1.28 netdiscover -i eth0



Fig. 1.29 netdiscover -p (passive mode)

Learning Outcomes:

- Installed and configured Kali Linux, Metasploitable 2 and Windows XP in VirtualBox and verified connectivity inside an isolated, authorised lab.
- Configured Sysmon and reviewed process-creation and other endpoint events in Windows Event Viewer.
- Used theHarvester to collect public email, host, subdomain and IP information for a domain through OSINT.
- Used Netdiscover to discover active hosts on a local network and identify their IP and MAC addresses.
- Understood that all scanning and testing must be limited to systems the tester owns or is authorised to test.

Ex. No. 2 **Date:** 01/07/2026
& 15/07/2026

Practice the basics of reconnaissance

Objective:

To practise the reconnaissance techniques used to build an authorised target profile: passive footprinting with search-engine operators (Google Dorking) and an OSINT workflow, domain and DNS information gathering with WHOIS and nslookup, subdomain discovery with Sublist3r, Internet-exposed asset discovery with Shodan, and active host, port, service and OS discovery with Nmap.

Tools:

Google search operators, WHOIS, nslookup (dnsutils), Sublist3r, Shodan, Nmap – run from Kali Linux.

Methodology:

Reconnaissance moves from passive (public data only, no packets sent to the target) to active (direct scanning). The experiment is carried out in eight parts (2.1 to 2.8); active steps are performed only on authorised lab systems.

2.1 Passive Footprinting

Principle: Footprinting gathers information about a target before touching its systems. Search-engine queries only contact the search engine, so they are passive and leave no trace in the target's logs.

1. Open Google in any browser (Chrome, Firefox or Edge).
2. Restrict results to a single domain with the site: operator and refine with file types, URL and title words.
3. Note the pages, documents and login/index pages that the domain exposes publicly.

```
site:example.com
site:example.com filetype:pdf
site:example.com inurl:login
site:example.com intitle:index
"exact phrase" site:example.com
```

Observation: site:amazon.com returned only pages indexed from that domain – the first step in mapping a target's public footprint.

2.2 Search Engine Reconnaissance / Google Dorking

Principle: Google Dorking (Google Hacking) uses advanced operators to filter Google's index for specific, often unintentionally exposed information such as documents, login pages and misconfigurations. It needs no installation and never contacts the target directly.

1. Use site: with filetype: to locate publicly indexed documents (e.g. PDFs).
2. Combine site: with intitle: to narrow results to pages on a particular topic.
3. Use inurl: to match words in the URL and intext: to match words in the page body.

```
site:owasp.org filetype:pdf cybersecurity
site:owasp.org intitle:"SQL Injection"
site:docs.aws.amazon.com inurl:iam
site:developer.mozilla.org intext:"JavaScript"
```

Other operators: - (exclude a term), " " (exact phrase), OR / | (either term) and cache: (Google's last cached snapshot).

Observation: Combining operators sharply narrows the results; exposed files and pages can be found before an attacker finds them.

2.3 Open-Source Intelligence (OSINT) Workflow

Principle: OSINT is the collection and analysis of publicly available information. A repeatable workflow moves from passive sources to active verification, and each tool in this experiment feeds the next stage.

1. Define the authorised scope (domain, IP range, lab hosts).
2. Passive footprinting: search-engine operators and WHOIS registration data.
3. DNS analysis: resolve A, MX and NS records with nslookup.
4. Subdomain discovery with Sublist3r and Internet-exposure lookup with Shodan.
5. Active scanning of authorised hosts with Nmap; verify, correlate and document all findings.

Observation: Public results can be stale, so findings must be verified against a second source before they are reported.

2.4 WHOIS Domain Enumeration

Principle: WHOIS is a command-line tool that retrieves registration information for a domain name or IP address – registrar, registration and expiry dates, name servers, status and (for IPs) the network range and owner.

1. Install WHOIS and verify the installation.
2. Look up a domain, then an IP address.
3. Query a particular registry server with -h when required.

```
sudo apt install whois
whois --version
whois --help
whois google.com
whois 8.8.8.8
whois -h whois.verisign-grs.com example.com
```

Observation: The domain lookup shows the registrar (MarkMonitor Inc.), creation date, status flags and name servers (NS1–NS4.GOOGLE.COM); the IP lookup shows the network range 8.8.8.0 – 8.8.8.255 owned by Google LLC, Mountain View, US.

2.5 DNS Analysis with nslookup

Principle: nslookup queries DNS servers to resolve domain names into IP addresses, mail servers (MX) and authoritative name servers (NS), and can also resolve an IP back to a name (reverse lookup).

1. Install dnsutils, which provides nslookup.
2. Resolve the domain, then query its MX and NS records.
3. Query a specific DNS server (8.8.8.8) and perform a reverse lookup.

```
sudo apt install dnsutils
nslookup example.com
nslookup -type=mx example.com
nslookup -type=ns example.com
nslookup example.com 8.8.8.8
nslookup 8.8.8.8
```

Observation: example.com resolved to 104.20.23.154 and 172.66.147.243 (plus an IPv6 address) and its name servers are hera.ns.cloudflare.com and elliott.ns.cloudflare.com; google.com's MX record is smtp.google.com (priority 10); microsoft.com uses Azure DNS name servers. The answers are marked non-authoritative because they come from a resolver's cache.

2.6 Subdomain Enumeration with Sublist3r

Principle: Sublist3r is an OSINT tool that discovers subdomains of a domain by querying public search engines and online sources (Google, Bing, Baidu, Yahoo, Netcraft, VirusTotal, ThreatCrowd, SSL certificates, PassiveDNS). Subdomains widen the known attack surface.

1. Update the package list and install Sublist3r; verify with --help.
2. Run Sublist3r against the target domain.
3. Save the results to a text file and display the file.

```
sudo apt update
sudo apt install sublist3r -y
sublist3r --help
sublist3r -d microsoft.com
sublist3r -d microsoft.com -o results.txt
cat results.txt
```

Observation: Sublist3r searched each source in turn and reported 260 unique subdomains for microsoft.com (e.g. www, apps, account, admin). The VirusTotal and DNSdumpster modules failed in this run, so results depend on which sources respond.

2.7 Internet Asset Discovery with Shodan

Principle: Shodan is a search engine for Internet-connected devices. It stores the banners and open ports it has already collected, so looking up an IP is passive – nothing is sent to the target.

1. Open shodan.io and search for the IP address (8.8.8.8).
2. Open the host page and review General Information and Open Ports.
3. Record the open ports, service banners and location data shown.

Observation: 8.8.8.8 resolves to dns.google, belongs to Google LLC (Mountain View, United States) and was last seen on 2026-07-28 with ports 53 (TCP/UDP, recursion enabled) and 443 open.

2.8 Network Scanning with Nmap

Principle: Nmap (Network Mapper) is a free, open-source scanner for host discovery, port scanning, service/version detection and OS fingerprinting. Unlike the passive tools above, Nmap sends packets to the target and can be seen in its logs and IDS, so it is run only against systems you own or are authorised to test.

1. Install Nmap and display the help/usage.
2. Host discovery: ping scan of the lab range (-sn, no port scan).
3. Port scanning: default top 1000 ports, selected ports (-p), all ports (-p-) and a SYN (stealth) scan (-sS, needs root).
4. OS detection (-O) and service/version detection (-sV); combine them, or use -A for the aggressive scan.

```
sudo apt install nmap -y
nmap --help
nmap -sn 192.168.56.0/24
nmap <target-ip>
nmap -p 22,80,443,587 <target-ip>
nmap -p- <target-ip>
sudo nmap -sS <target-ip>
sudo nmap -O <target-ip>
nmap -sV <target-ip>
sudo nmap -sS -sV -O <target-ip>
sudo nmap -A <target-ip>
```

Observation: The ping scan of 10.0.2.0/24 found 3 live hosts out of 256. The service scan showed ports 22 (OpenSSH 8.2p1), 80 (Apache httpd 2.4.41), 139/445 (Samba smbd 4), 8009 (Apache Jserv) and 8080 (Apache Tomcat 9.0.7) open. OS detection returned only a fingerprint with no exact match, so OS guesses are indicative rather than certain. The screenshots were captured on the manual's example targets; the commands above are to be run only on an authorised lab host.

Screenshots:

2.1 Passive Footprinting

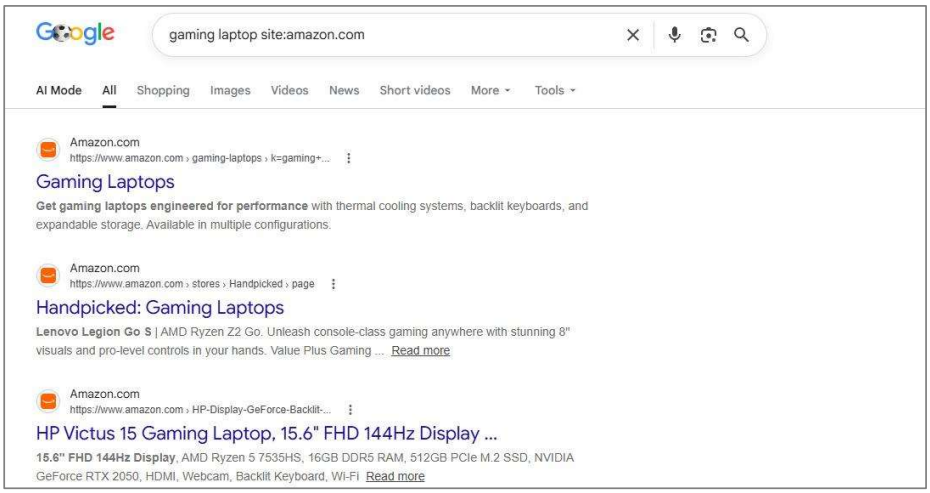


Fig. 2.1 Google query site:amazon.com

2.2 Google Dorking

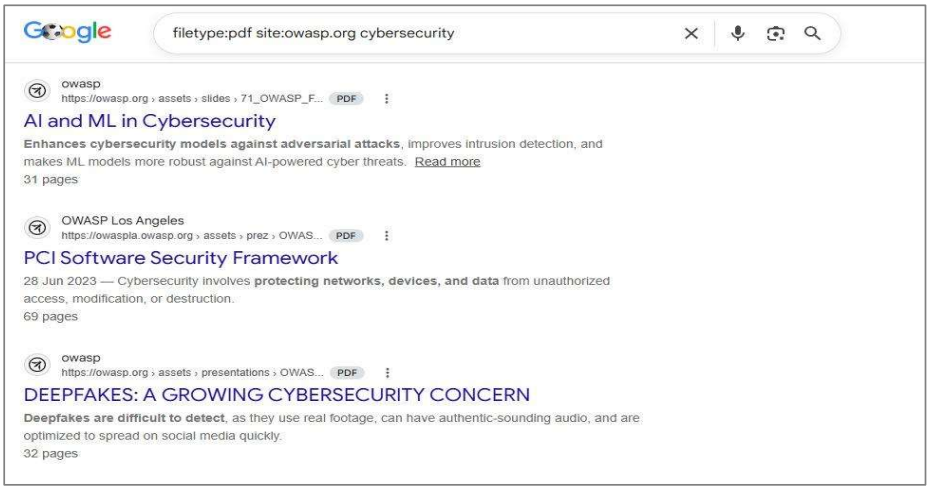


Fig. 2.2 filetype:pdf site:owasp.org cybersecurity

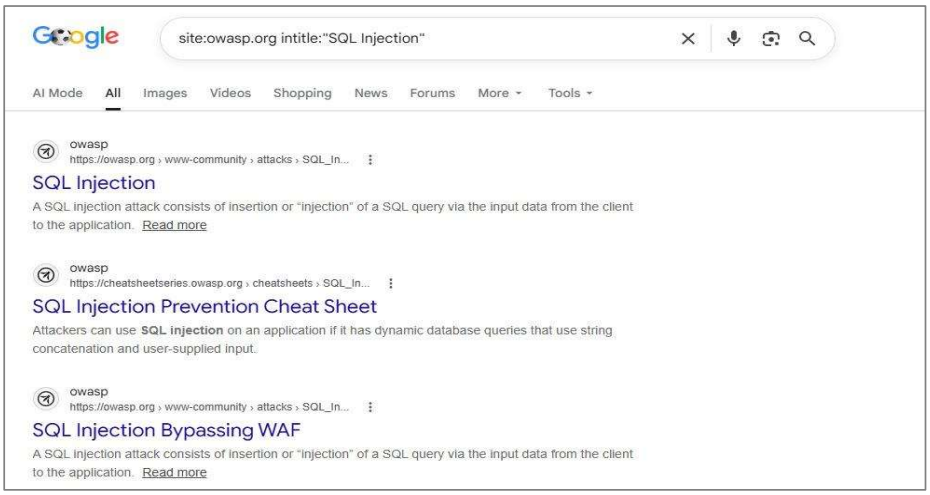


Fig. 2.3 site:owasp.org intitle:"SQL Injection"

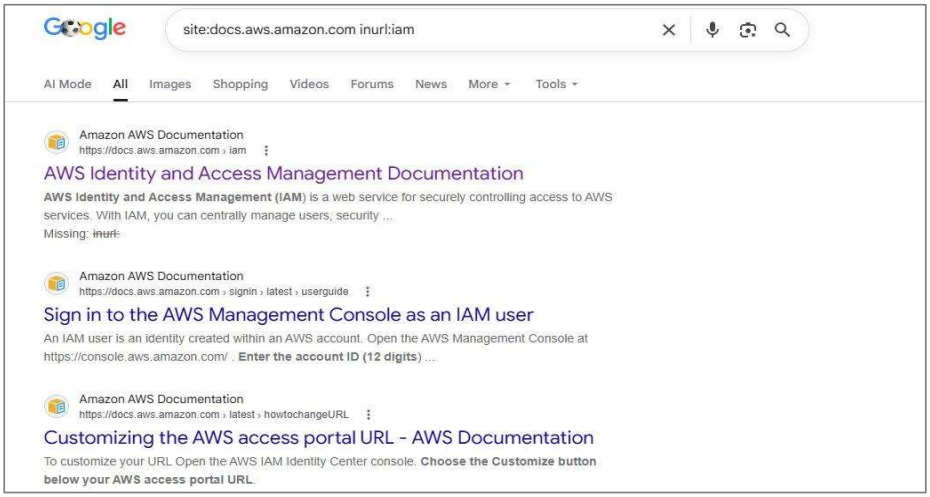


Fig. 2.4 site:docs.aws.amazon.com inurl:iam

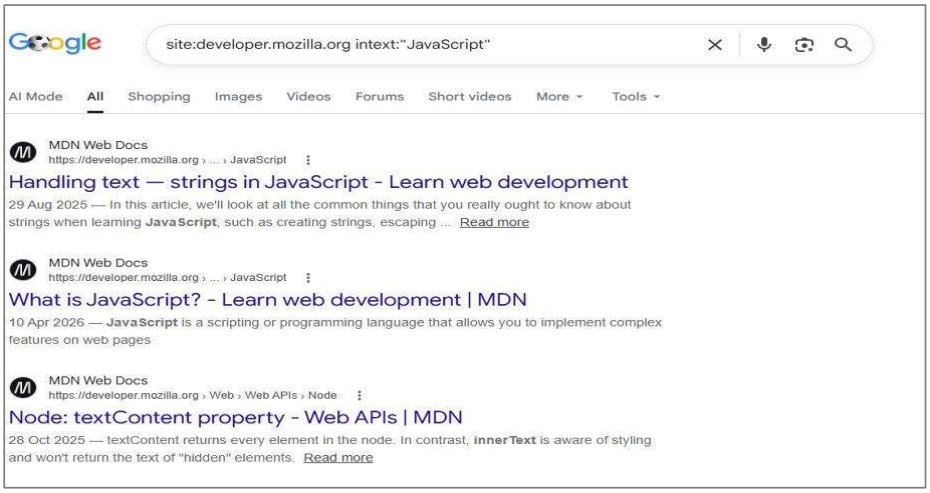


Fig. 2.5 site:developer.mozilla.org intext:"JavaScript"

2.3 OSINT Workflow

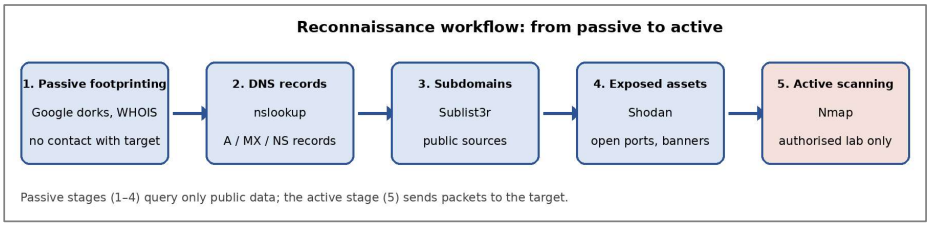


Fig. 2.6 Reconnaissance workflow from passive to active

2.4 WHOIS Domain Enumeration



Fig. 2.7 whois --version



Fig. 2.8 whois google.com



Fig. 2.9 whois 8.8.8.8



Fig. 2.10 whois -h whois.verisign-grs.com (specific WHOIS server)

2.5 DNS Analysis with nslookup

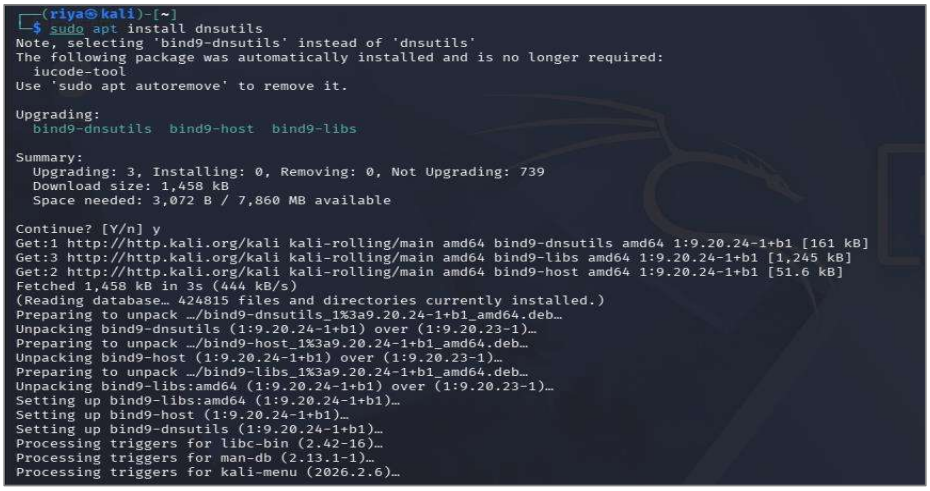


Fig. 2.11 Installing dnsutils

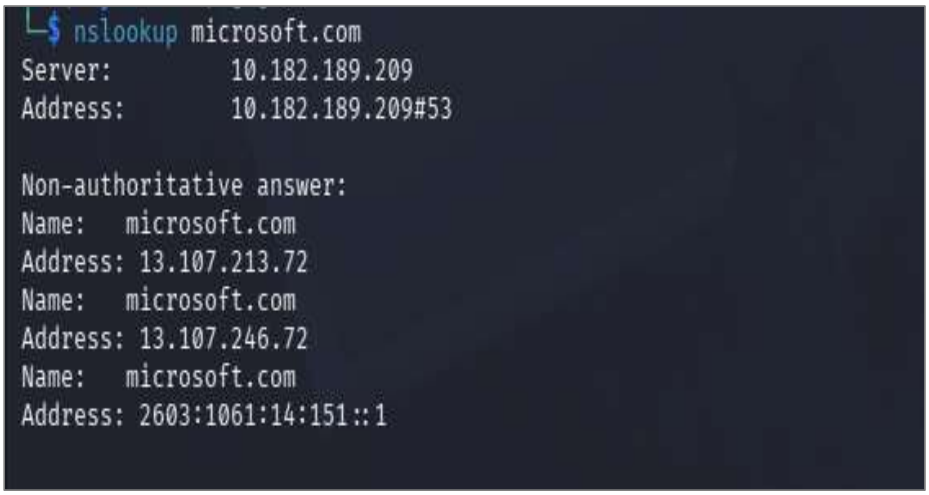


Fig. 2.12 nslookup example.com / microsoft.com



Fig. 2.13 nslookup -type=mx

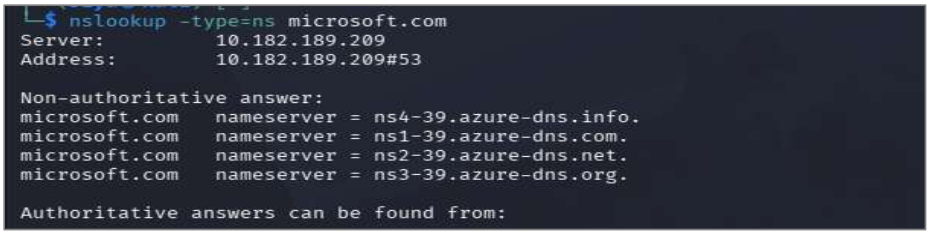


Fig. 2.14 nslookup -type=ns



Fig. 2.15 nslookup example.com 8.8.8.8

2.6 Subdomain Enumeration with Sublist3r



Fig. 2.16 Installing Sublist3r



Fig. 2.17 sublist3r --help

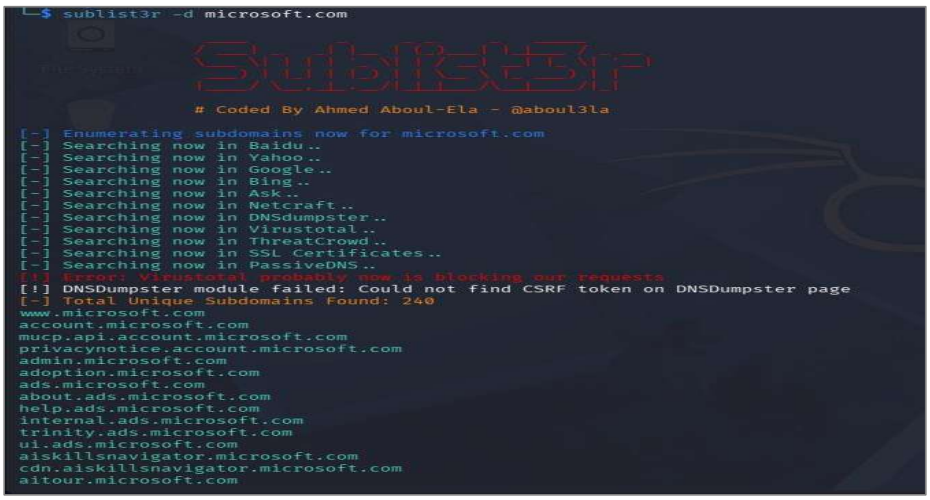


Fig. 2.18 sublist3r -d microsoft.com



Fig. 2.19 sublist3r -d microsoft.com -o results.txt



Fig. 2.20 cat results.txt

2.7 Internet Asset Discovery with Shodan

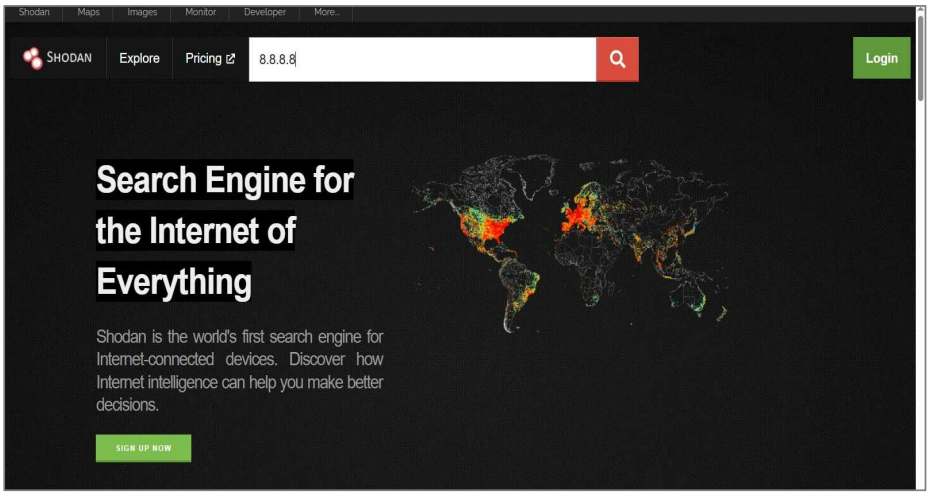


Fig. 2.21 Shodan search for 8.8.8.8

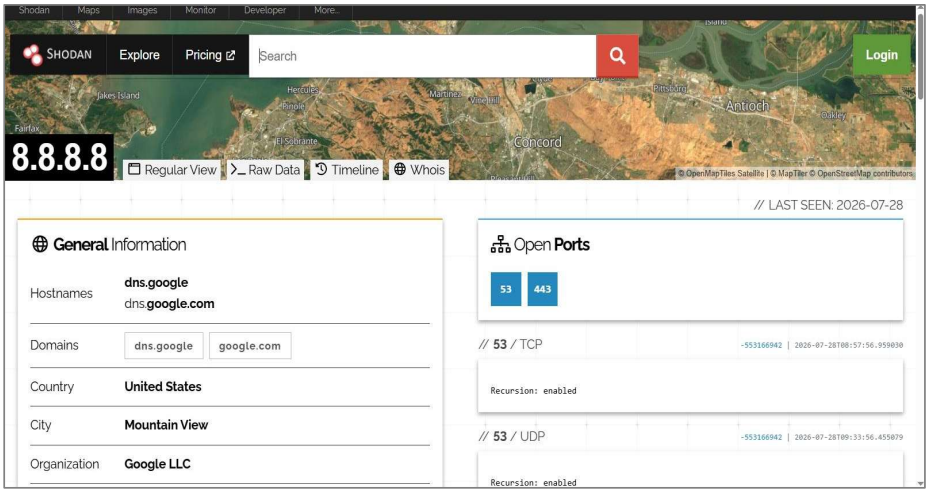


Fig. 2.22 Shodan host page for 8.8.8.8 – general information and open ports

2.8 Network Scanning with Nmap

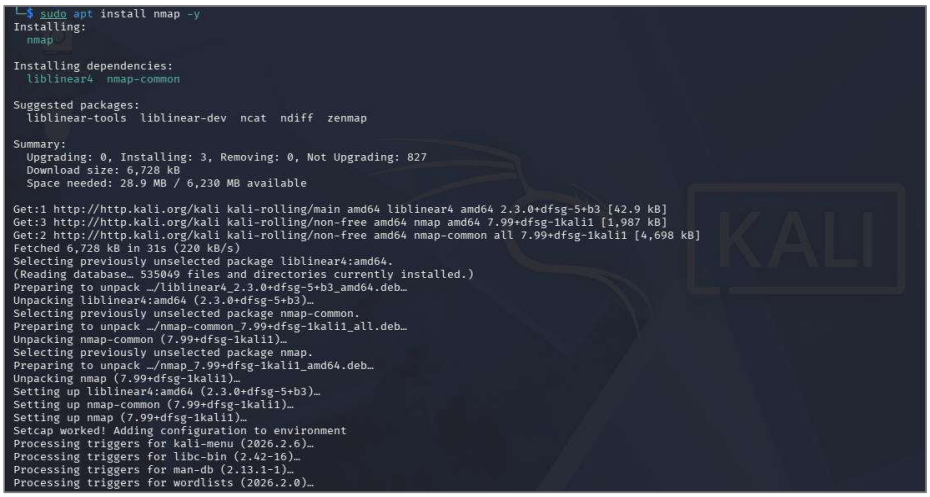


Fig. 2.23 Installing Nmap

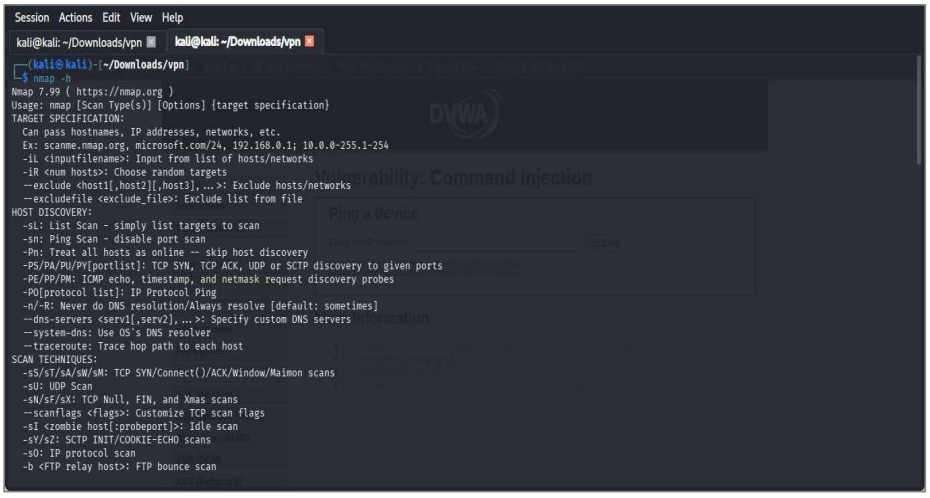


Fig. 2.24 nmap --help

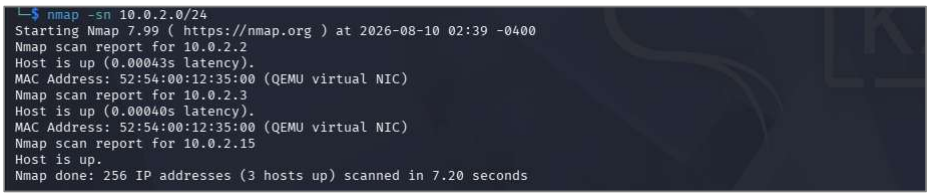


Fig. 2.25 nmap -sn 10.0.2.0/24 (ping scan)

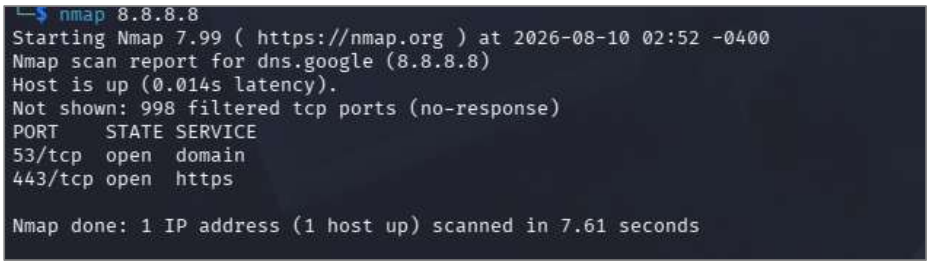


Fig. 2.26 nmap 8.8.8.8 (basic port scan)

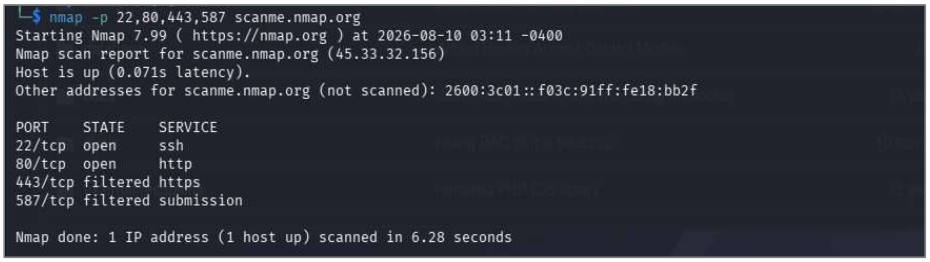


Fig. 2.27 nmap -p 22,80,443,587 scanme.nmap.org

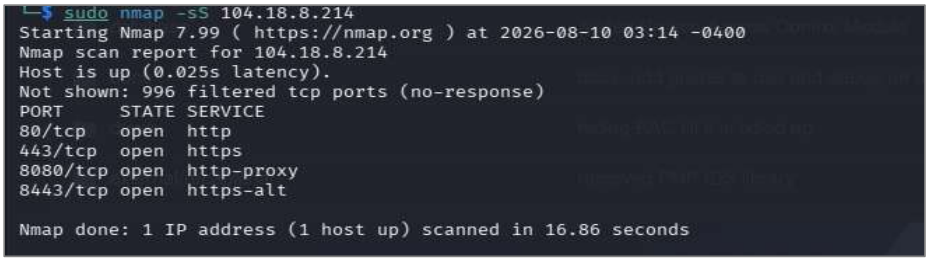


Fig. 2.28 sudo nmap -sS (SYN scan)

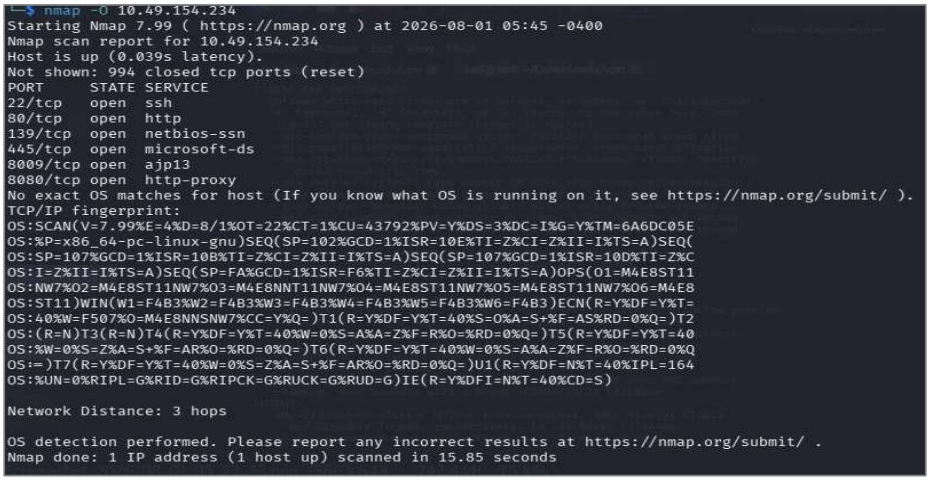


Fig. 2.29 nmap -O (OS detection)

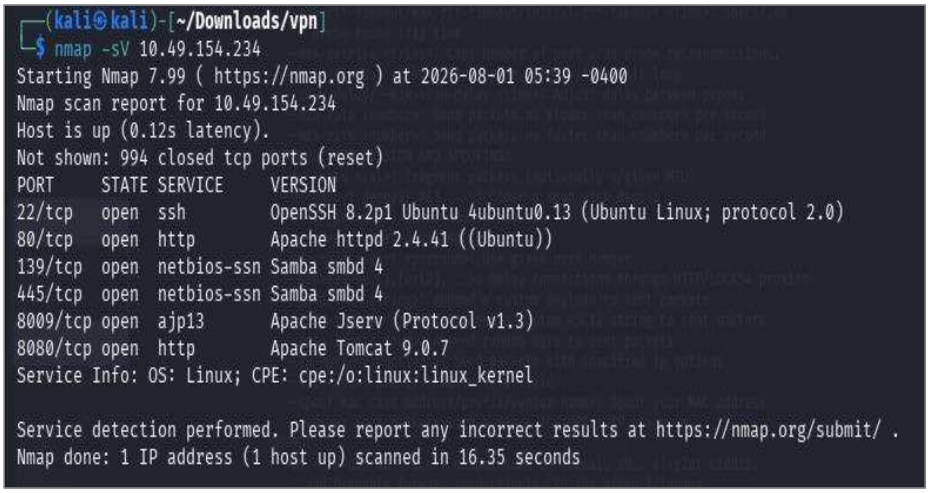


Fig. 2.30 nmap -sV (service/version detection)

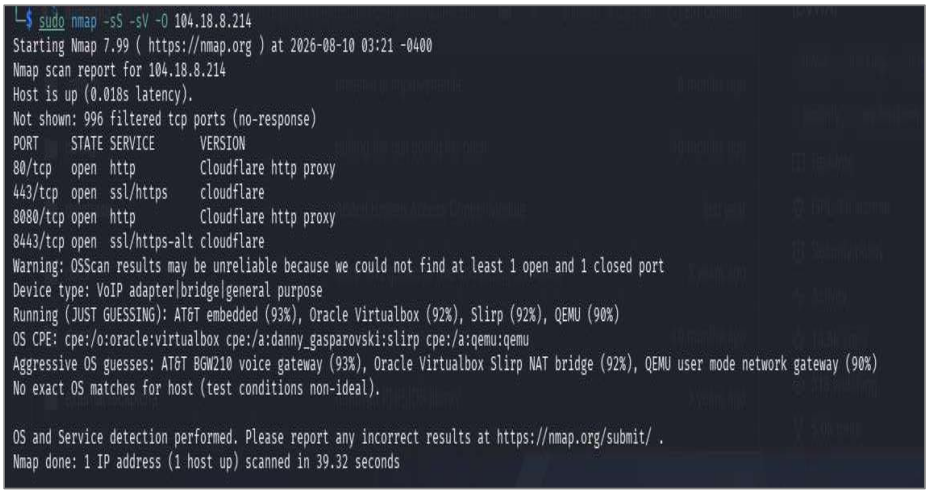


Fig. 2.31 sudo nmap -sS -sV -O (combined scan)

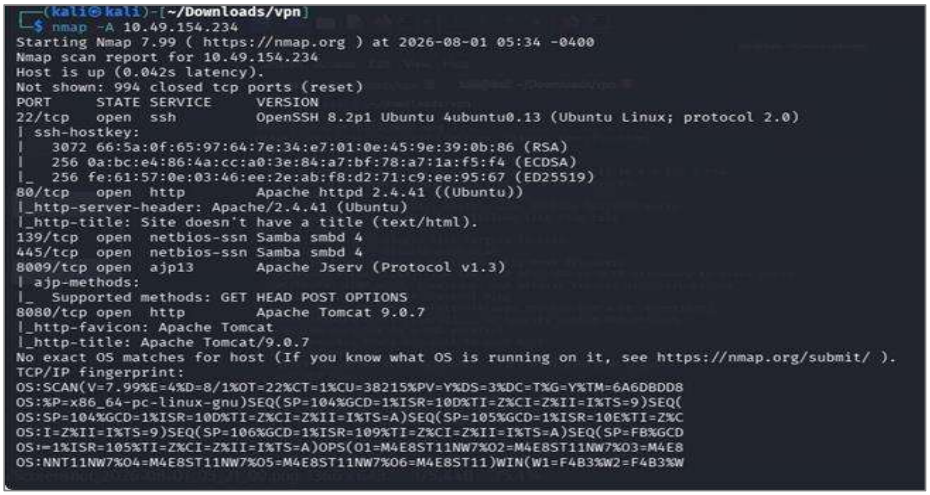


Fig. 2.32 nmap -A (aggressive scan)

Learning Outcomes:

- Used site:, filetype:, intitle:, inurl: and intext: operators to perform passive footprinting and Google Dorking without contacting the target.
- Retrieved registration, name-server and network-owner details with WHOIS and interpreted DNS A, MX and NS records with nslookup.
- Discovered subdomains with Sublist3r and reviewed an IP's publicly exposed ports and banners on Shodan.
- Performed host discovery, port scanning, service/version detection and OS fingerprinting with Nmap and interpreted the results.
- Understood the difference between passive and active reconnaissance, and that active scanning must be limited to authorised systems.

Ex. No. 3 **Date:** 22/07/2026

Using FOCA / SearchDiggity tools, extract metadata and expand the target list

Objective:

To extract hidden metadata from publicly available documents and use search-based discovery to expand the list of known public assets of an authorised target: metadata analysis with FOCA (documents, author names, usernames, software, folders, printers and network/domain hints) and automated search-engine reconnaissance with SearchDiggity (Google Hacking Database style queries), followed by correlation and de-duplication of the newly found names, hosts and files.

Tools:

FOCA (ElevenPaths, v3.4.7.1) with Microsoft SQL Server Express, SearchDiggity (Bishop Fox), Google / Bing search – run from a Windows lab host.

Methodology:

Public documents and search results are collected, hidden metadata is extracted, and newly identified names, hosts and files are correlated and de-duplicated into an expanded target list. Both tools query only search engines and public files, so the activity is passive; documents are downloaded only from an authorised or example domain. The experiment is carried out in two parts (3.1 and 3.2).

3.1 Metadata Analysis with FOCA

Principle: Metadata is descriptive information stored inside files and can include author names, usernames, software versions, creation details, computer names, e-mail addresses and internal file paths. FOCA (Fingerprinting Organizations with Collected Archives) is an OSINT tool that finds public PDF, Word, Excel and PowerPoint documents of a domain through search engines, downloads them and extracts this hidden information, so that an organisation can see what its public files reveal.

1. Download the latest FOCA release (v3.4.7.1 ZIP) from the official ElevenPaths GitHub Releases page.
2. Download and install Microsoft SQL Server Express, which FOCA needs for its project database: run the installer, select Basic, accept the licence and finish the installation.
3. Extract the FOCA ZIP file and run FOCA.exe.
4. Create a new project: enter a project name, the authorised/example target in Domain website, any alternative domains and the folder where documents will be saved, then click Create. A "Project saved successfully" message confirms the project.
5. Click Search All so that FOCA queries Google, Bing and DuckDuckGo for publicly indexed documents of the domain (doc, docx, xls, xlsx, ppt, pptx, pdf, odt, ods and similar extensions).
6. Select the documents in the results list, right-click and choose Download All to save the permitted files locally; the Download column changes to a green dot when a file has been fetched.
7. Right-click and choose Extract All Metadata to pull hidden information such as author names, software versions and internal file paths from the downloaded files.

8. Choose Analyze All Metadata so that FOCA correlates the findings across all documents – usernames, software, printers, folders and the possible network/domain structure (shown under Network, Domains and Metadata in the project tree).
9. Record the sensitive items found (people, internal paths, software versions), add the new names and hosts to the target list, and note the remediation: sanitise documents before publishing them.

```
site:<domain> (filetype:pdf OR filetype:doc OR filetype:docx OR filetype:xls OR filetype:xlsx OR filetype:ppt OR filetype:pptx)    # the type of query FOCA sends to the search engines
```

Observation: In the manual's example run (project for the domain xfinity.com, FOCA 3.4.6.0 project window), FOCA listed more than 18 public documents and web resources (docx files and linked media files of between about 16 KB and 6.6 MB), all of which showed the green downloaded mark after Download All. Metadata can reveal people, software and internal naming, but not every metadata field is accurate and not every indexed file is still online, so findings must be verified before they are reported. The screenshots were taken from the lab manual's example; the same steps are to be run only on an authorised domain.

3.2 SearchDiggity for Search-Based Exposure

Principle: SearchDiggity is the Windows GUI front-end of the Bishop Fox Google Hacking Diggity Project. It automates large sets of specialised search queries (the Google Hacking Database for Google, and a Bing hacking database for Bing) through its GoogleDiggity, BingDiggity and ShodanDiggity modules, and other modules such as DLPDiggity and PortScanDiggity. The queries can be limited to one domain, so public pages, files and references of the target are listed in one run. The results are search-engine data and can contain false positives, so they must be verified.

1. Download SearchDiggity from the Bishop Fox Google Hacking Diggity Project page, install it on the Windows lab host and start the application.
2. Open Settings of the module to be used. By default the Google and Bing modules collect results by scraping; optionally disable the scraper and enter an API key (Google, Bing or Shodan) to avoid bot-detection problems.
3. Open the Google (or Bing) tab, enter the authorised domain in the site/domain box and select safe, footprinting-type query categories from the query list (for example, footprinting, error messages and directory listings).
4. Click Scan and let the queries run; the results grid lists each matching page with the query that found it, its title and URL.
5. Review the hits, discard false positives, and record newly identified hosts, subdomains, files and references.
6. Merge these entries with the names and hosts obtained from FOCA in 3.1, remove duplicates and verify each item against a second source to build the expanded target list.

Observation: Search-based discovery only reads what search engines have already indexed; it must not be used to open restricted content or bypass access controls. Results depend on the search engine and API access available when the tool is run (search-engine APIs have changed over the years), so the number of hits can differ between runs and always needs verification. Together, FOCA (hidden names and paths inside documents) and SearchDiggity (indexed pages and files) widen the asset inventory of the target.

Screenshots:

3.1 Metadata Analysis with FOCA

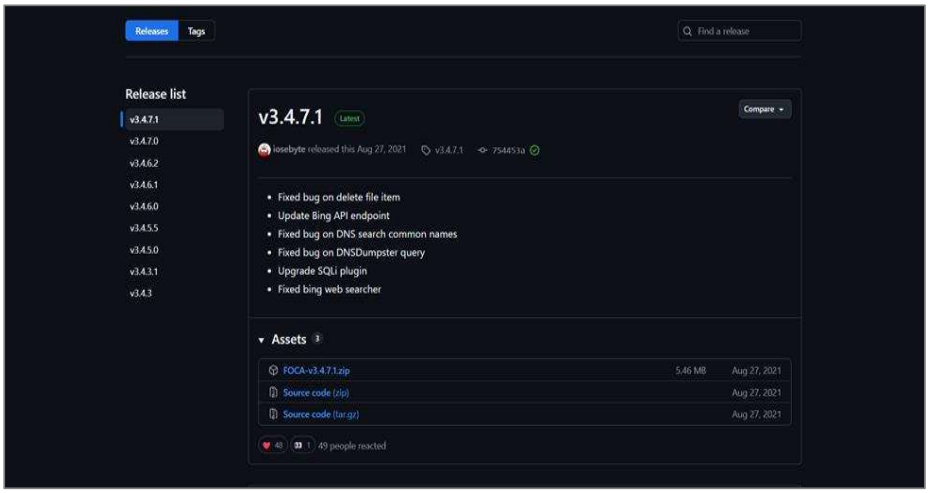


Fig. 3.1 FOCA v3.4.7.1 on the ElevenPaths GitHub Releases page

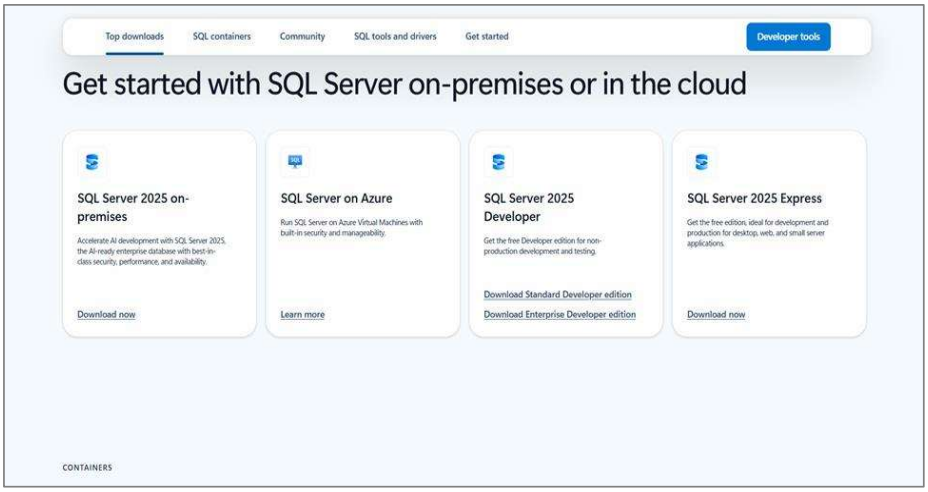


Fig. 3.2 Microsoft SQL Server downloads page (SQL Server Express)

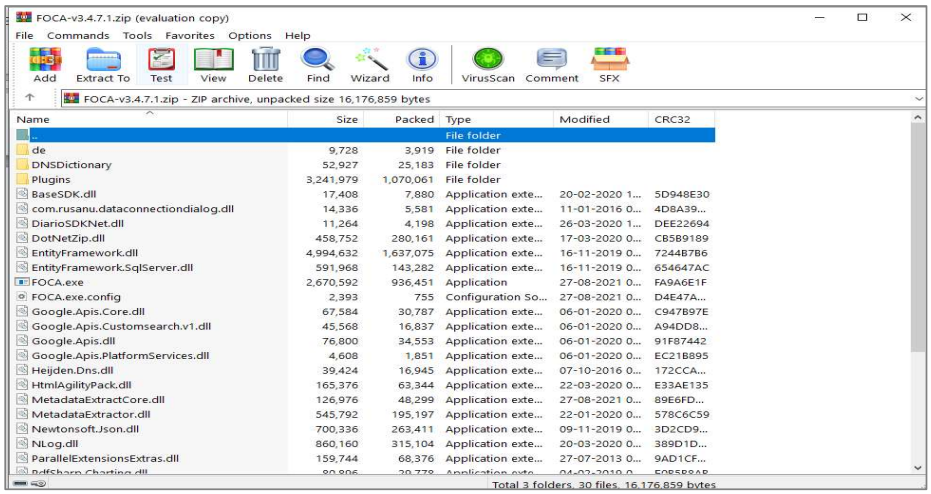


Fig. 3.3 Extracted FOCA ZIP file containing FOCA.exe

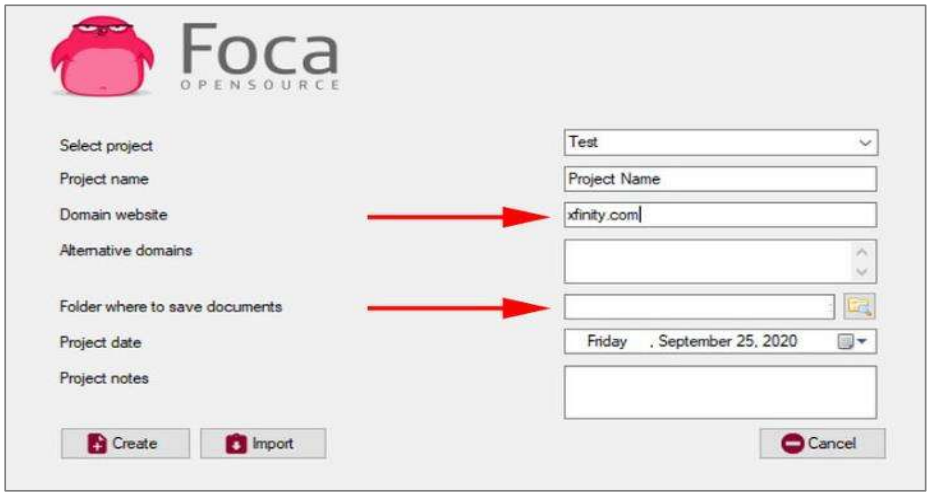


Fig. 3.4 New FOCA project – domain website and document folder

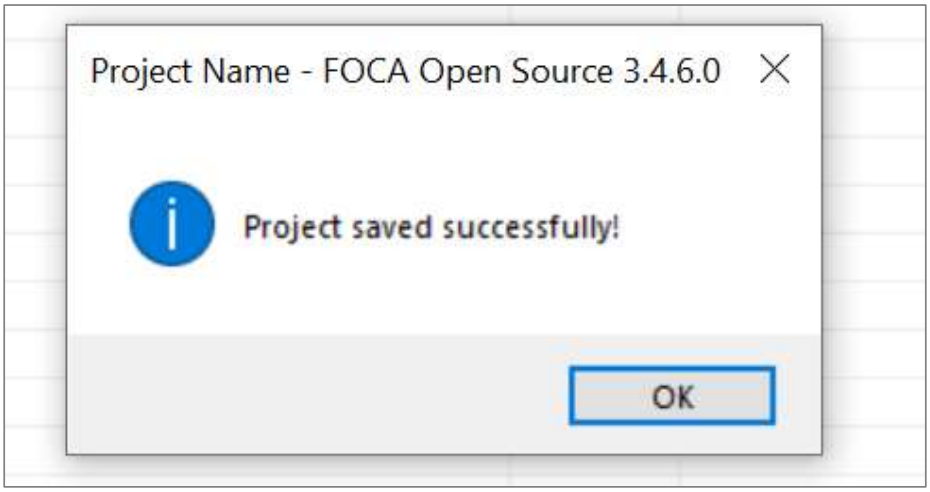


Fig. 3.5 Project saved successfully

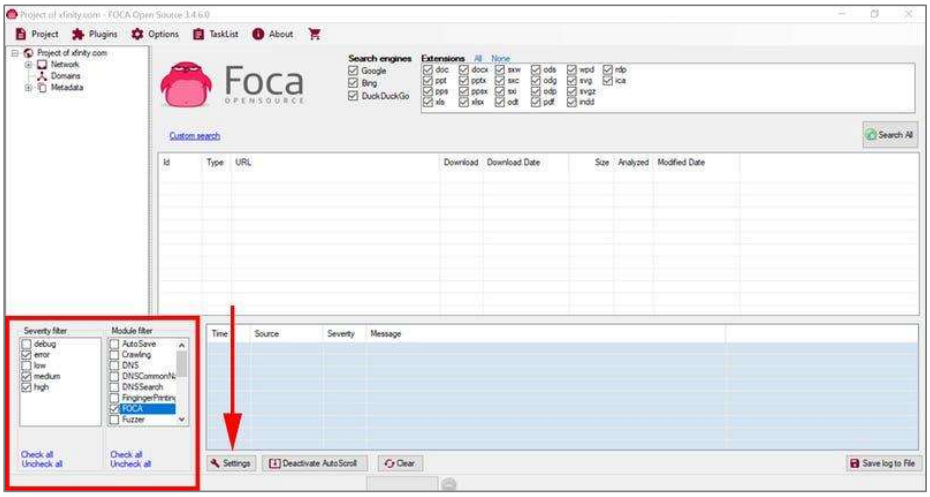


Fig. 3.6 FOCA project window with the Search All option

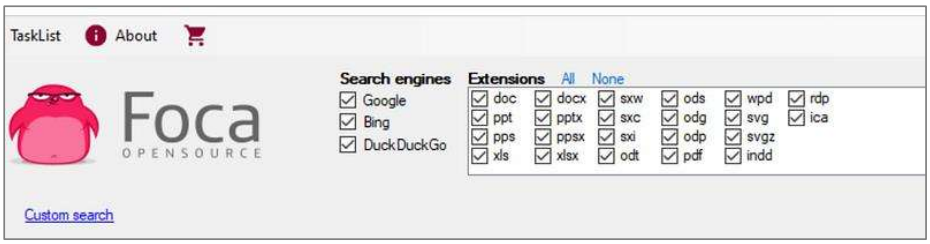


Fig. 3.7 Search engines and file extensions selected for the search

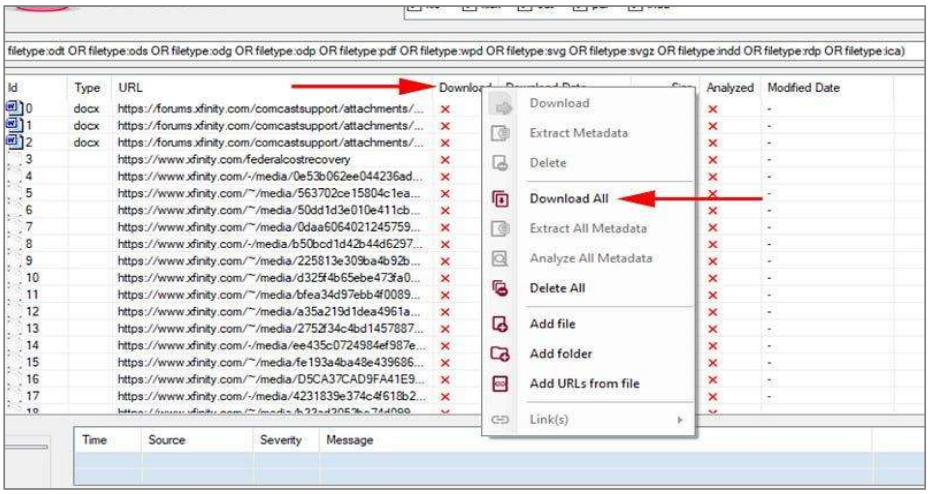


Fig. 3.8 Right-click menu – Download All, Extract All Metadata, Analyze All Metadata

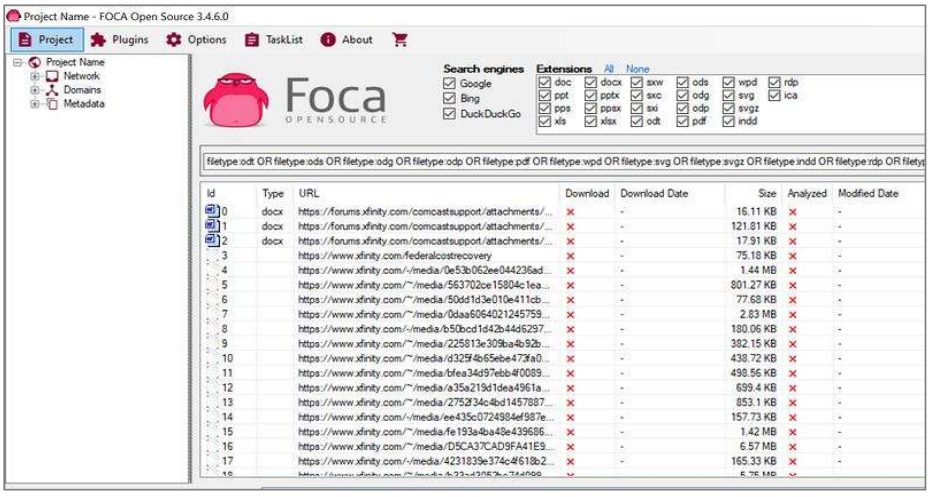


Fig. 3.9 Documents discovered for the domain (before download)

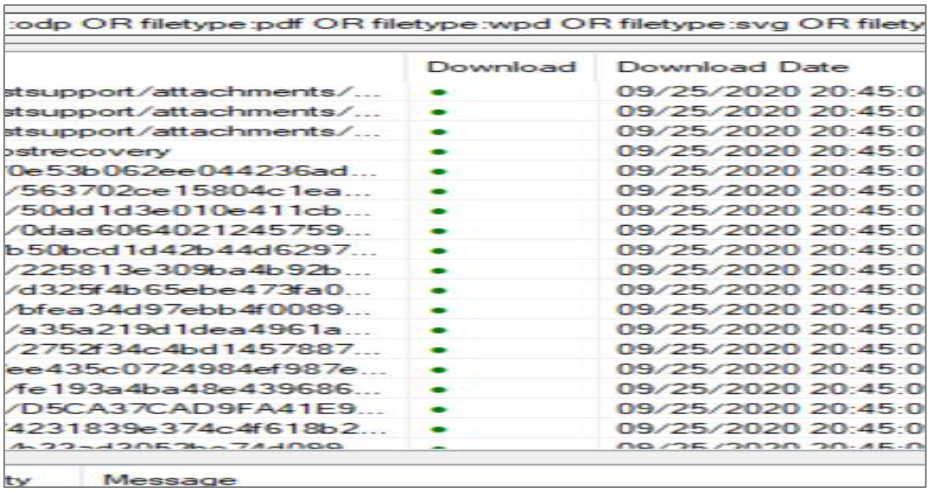


Fig. 3.10 Documents downloaded (green mark in the Download column)

Learning Outcomes:

- Installed FOCA with SQL Server Express, created a project for a domain and used it to find, download and analyse public documents.
- Extracted metadata (authors, usernames, software versions, folders and file paths) and identified how it leaks internal information.
- Used SearchDiggity to automate search-engine queries for a domain and to find additional public pages, hosts and files.
- Correlated and de-duplicated the results of both tools into an expanded target list and understood that search results must be verified.
- Understood that these techniques are passive and must be used only on authorised or example domains, and that documents should be sanitised before publication.

Ex. No. 4 **Date:** 29/07/2026

Aggregate information from public databases using online free tools like Paterva's Maltego

Objective:

To aggregate publicly available information about an authorised target domain and visualise the relationships between Internet assets using Maltego, a graphical link-analysis tool: representing data as entities, running transforms to discover DNS names, e-mail addresses and registration contacts, tracing the links between them, and enriching an IP address found in the graph with the Shodan search engine.

Tools:

Maltego Community Edition (Paterva / Maltego Technologies) with Java 17, Shodan – run from a lab host with Internet access.

Methodology:

Start with an authorised domain as the first entity, run passive transforms to expand the graph, inspect the relationships and validate the important nodes before reporting. Maltego and Shodan operate on data that is already public, so no packets are sent to the target's own systems. The experiment is carried out in two parts (4.1 and 4.2).

4.1 Relationship Mapping with Maltego

Principle: Open Source Intelligence (OSINT) is the collection and analysis of publicly available information. Maltego is a visual link-analysis platform that shows the results as a connected graph instead of a flat list. An Entity is a single piece of data (Domain, DNS Name, E-mail Address, Person, IP Address); a Transform is a query that takes one entity and returns new, related entities; and the Graph is the growing map of entities and the links between them, which makes shared infrastructure and clusters easy to see.

1. Register a free Maltego Community Edition account at maltego.com using a valid e-mail address, and verify the e-mail through the confirmation link before logging in.
2. After login, click Maltego Graph (Desktop) on the dashboard, download the Windows installer and run it; Maltego needs Java 17, so the setup first installs the Eclipse Temurin JRE, then installs Maltego (keep Create Desktop Shortcut ticked).
3. Open Maltego, choose the Maltego ID activation (online) and log in with the verified Maltego account; accept the licence agreement and authenticate in the browser until the "Authentication complete" page appears.
4. Continue the configuration wizard: select the data sources with the default Utilities transforms, download them, choose the default web browser and click Finish; open a blank graph.
5. Know the interface: Entity Palette (left) – drag entities onto the canvas; Graph Canvas (centre) – entities and links; Detail View / Property View / Transform Hub (right) – entity properties and transforms; Machines tab – bundled multi-step transform sequences.

6. Create a new graph (File > New Graph) and drag a Domain entity from the Entity Palette onto the canvas.
7. Double-click the entity and type the authorised training domain (ssn.edu.in in the manual's example).
8. Right-click the Domain entity > Run Transform > "To DNS Name – [DNS]" to discover the linked name servers, mail servers and sub-domains.
9. Right-click the Domain entity again > Run Transform > "To Email Address" to discover related e-mail addresses.
10. Run the "Persons from Domain" / WHOIS-based transform to add the registrant, administrative and technical contact entities (organisation, address, e-mail, person).
11. Review the related domain, DNS, IP, e-mail and person entities, remove irrelevant or duplicate nodes, and note the most important relationships.

Observation: The DNS transform linked the domain to its name servers (tdns, pdns and sdns), its web host (www) and its Google mail servers (the aspmx MX records); the Output window showed that a zone-transfer attempt was refused, which is the correct, secure behaviour. The e-mail transform returned four related e-mail addresses, which show the naming pattern used by the organisation, and the persons/WHOIS transform added the registrant, administrative and technical contact entities. An entity is a node, a transform is the operation that finds related nodes, and third-party or historical data must be validated before it is reported. The screenshots were captured on the manual's example domain; the same steps are to be run only on a domain you own or are authorised to investigate, and the e-mail and name data found is public but must be handled with care.

4.2 Enriching Findings with Shodan

Principle: Shodan is a search engine for Internet-connected devices. For an IP address it shows the open ports, service banners, hostnames, organisation and location that its crawlers have observed, so it can be used to cross-verify and expand an IP address found in the Maltego graph without touching the target directly.

1. Choose one IP address entity from the Maltego graph (8.8.8.8 is used in the manual).
2. Open shodan.io, type the IP address in the search box and press Enter to open its host page.
3. Record the open ports, service banners and location data shown, and compare them with the Maltego findings.

Observation: The Shodan host page for 8.8.8.8 showed the hostnames dns.google and dns.google.com, the organisation Google LLC in Mountain View, United States, and open ports 53 (TCP and UDP, DNS recursion enabled) and 443. This matches the public-DNS role of the address and the port scan of the same address in Ex. 2. Search-engine data may be out of date, so the entry's last-seen date should be checked.

Screenshots:

4.1 Relationship Mapping with Maltego

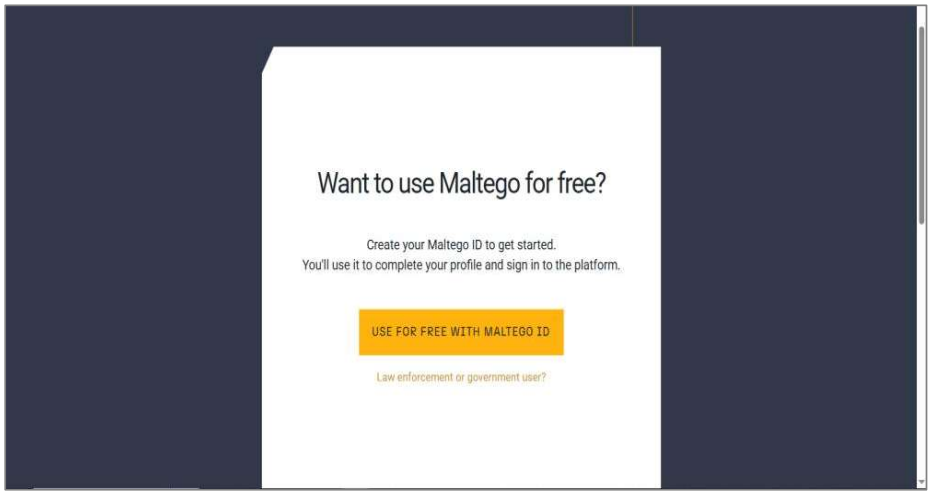


Fig. 4.1 Maltego Community Edition registration page

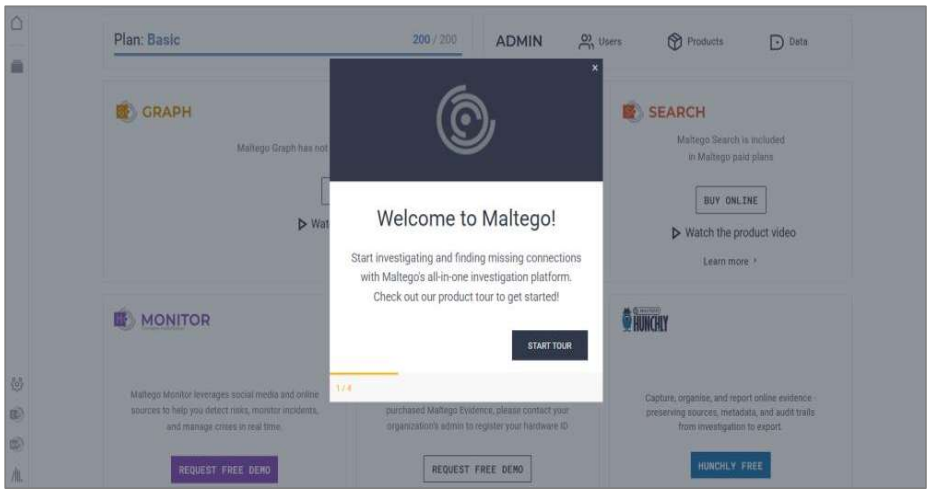


Fig. 4.2 Welcome to Maltego – account dashboard

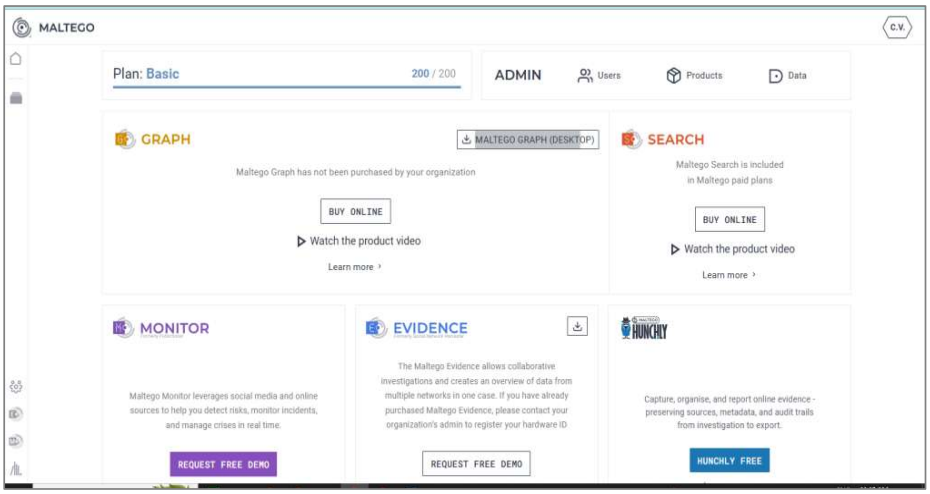


Fig. 4.3 Dashboard with the Maltego Graph (Desktop) download option

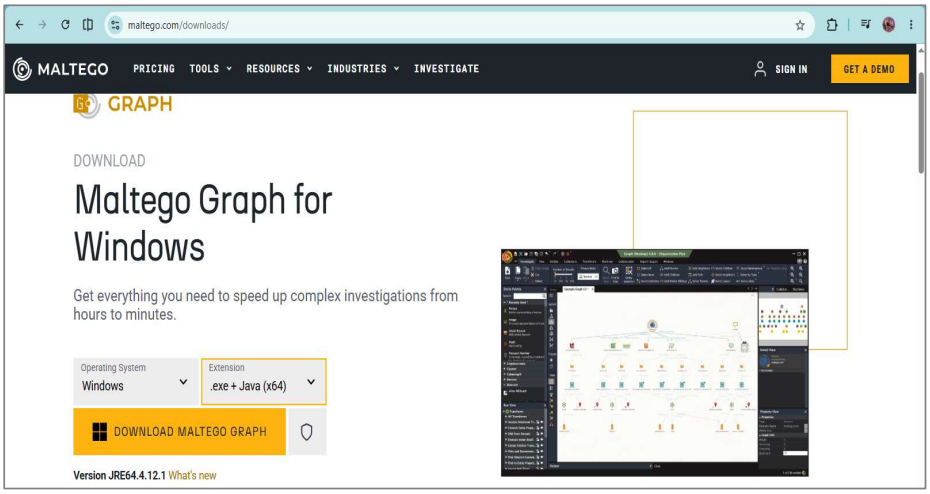


Fig. 4.4 Maltego Graph for Windows download page

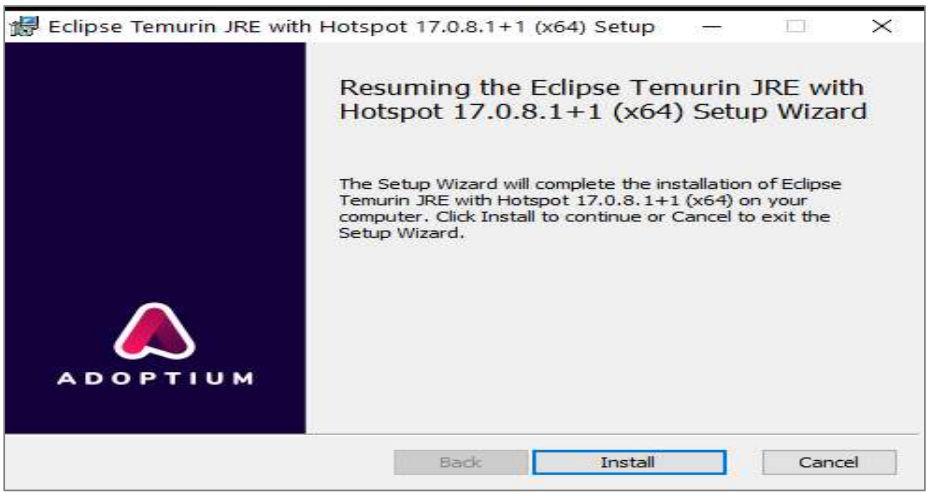


Fig. 4.5 Eclipse Temurin JRE 17 installation (Java prerequisite)

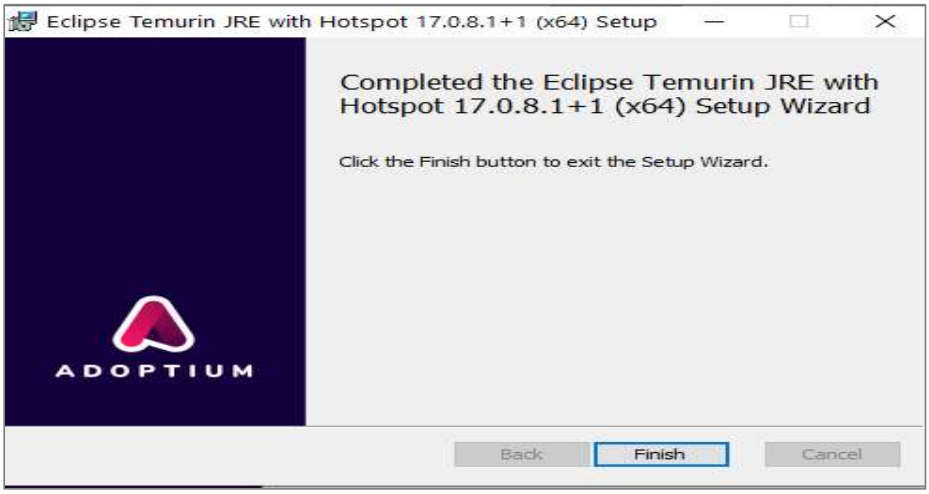


Fig. 4.6 Java installation completed

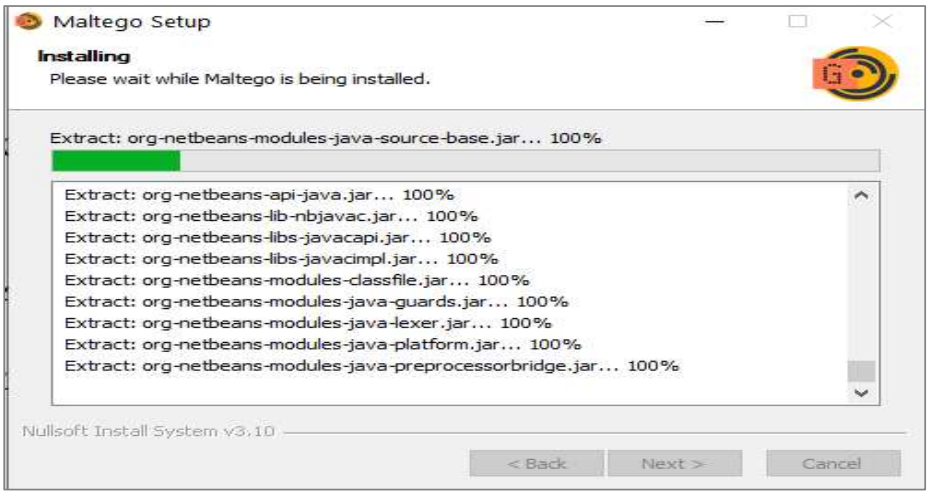


Fig. 4.7 Maltego installer extracting files

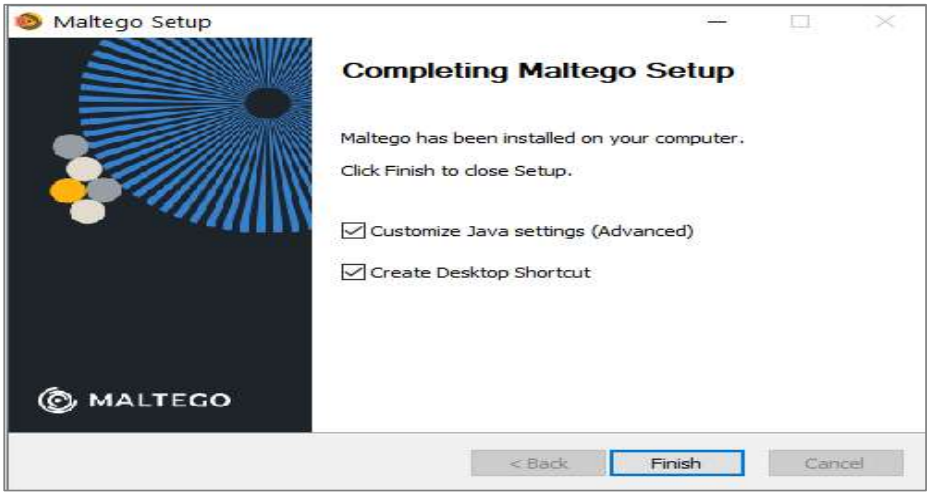


Fig. 4.8 Completing Maltego Setup



Fig. 4.9 Configure Maltego – activation options



Fig. 4.10 Maltego ID activation (online)



Fig. 4.11 Licence agreement

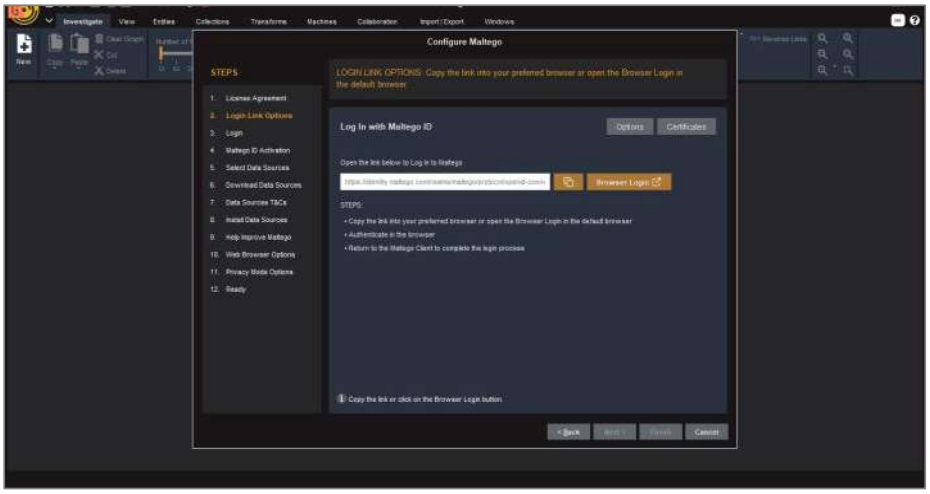


Fig. 4.12 Log in with Maltego ID – browser login link

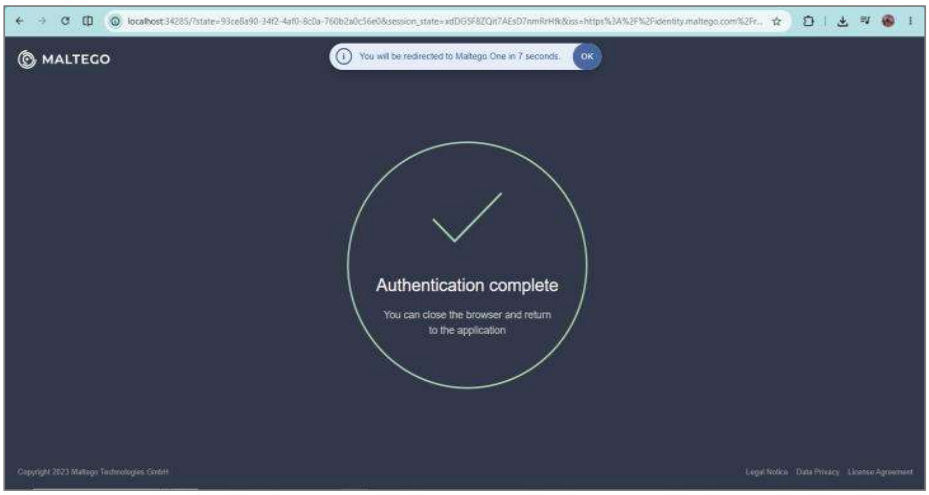


Fig. 4.13 Browser authentication complete

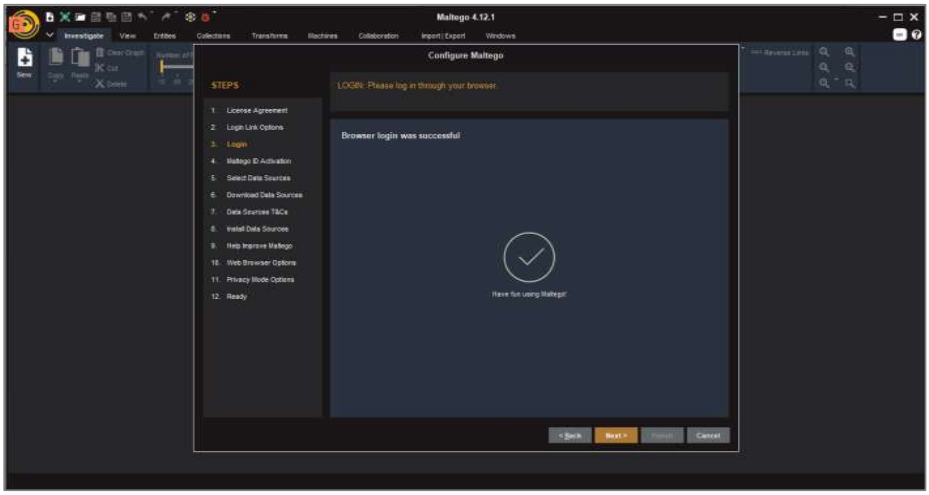


Fig. 4.14 Browser login successful in Maltego

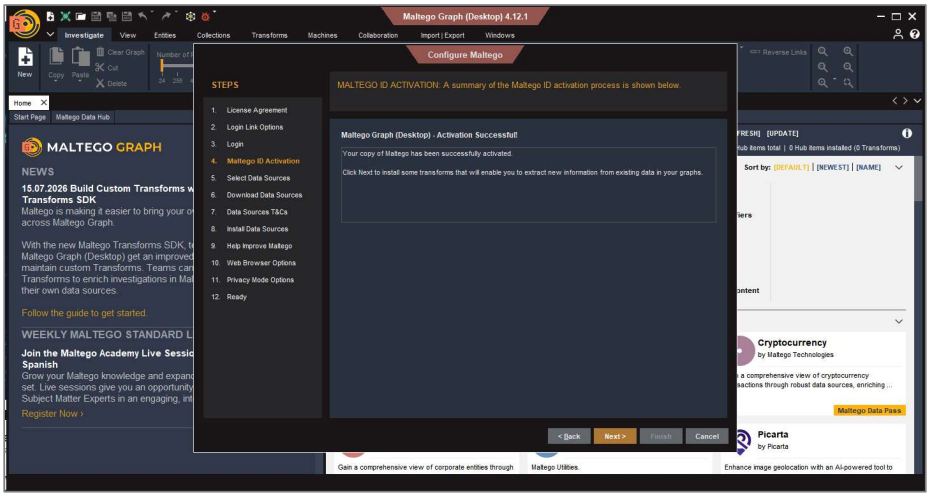


Fig. 4.15 Maltego Graph (Desktop) activation successful

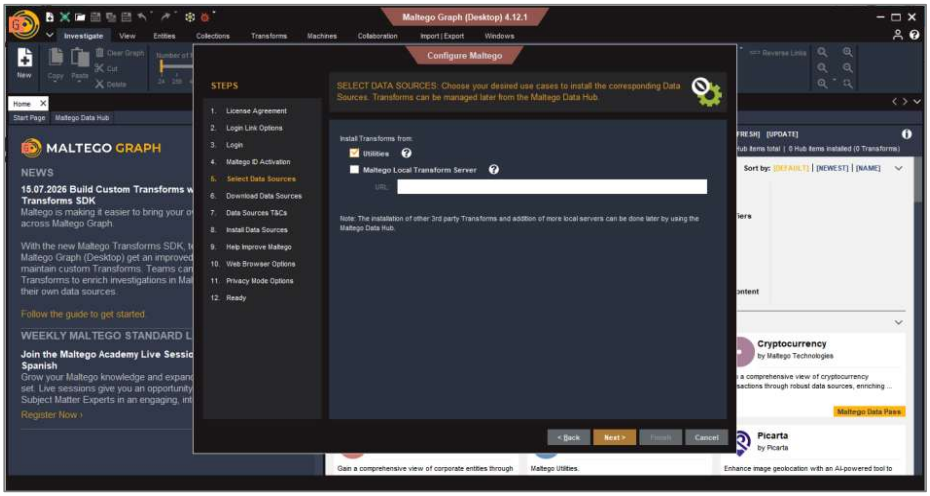


Fig. 4.16 Selecting the data sources (Utilities transforms)

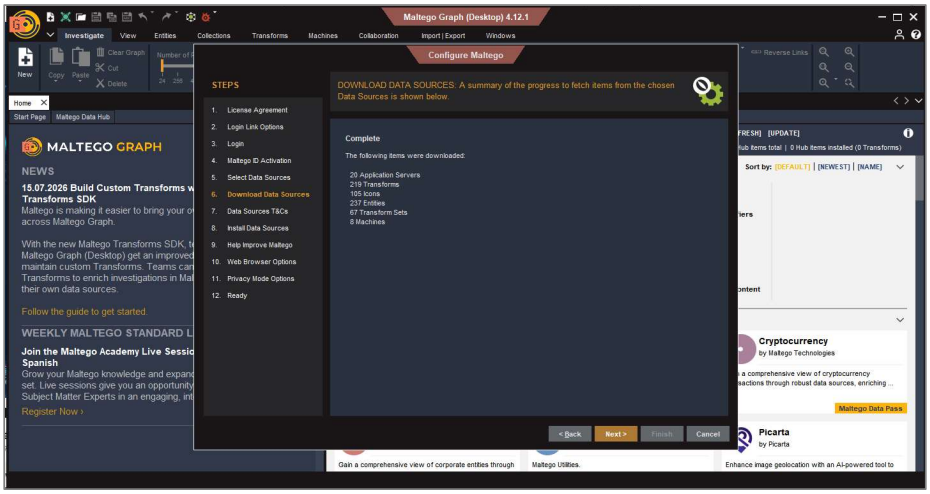


Fig. 4.17 Data sources downloaded

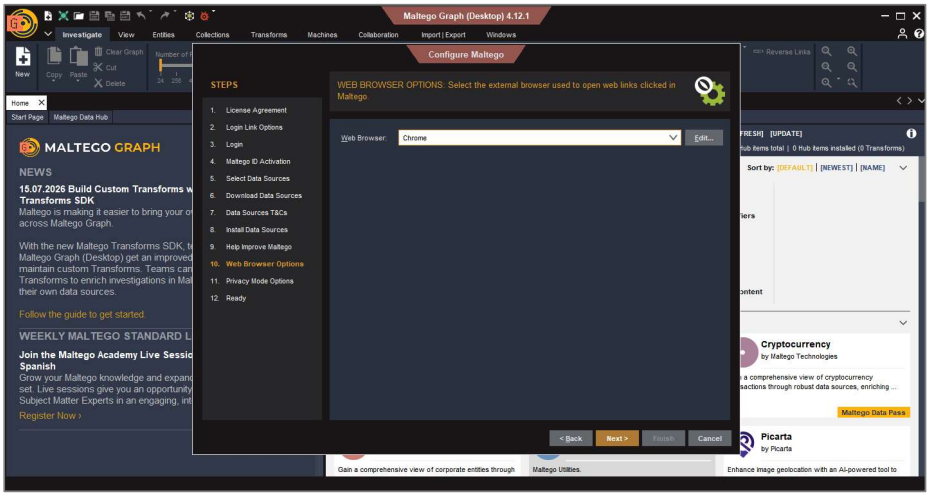


Fig. 4.18 Web browser option

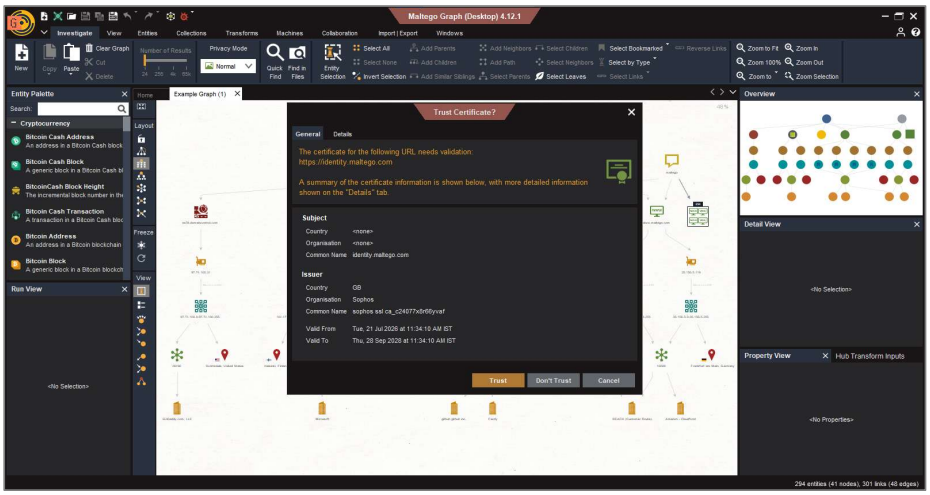


Fig. 4.19 Maltego interface with an example graph (Entity Palette, canvas, Detail View)

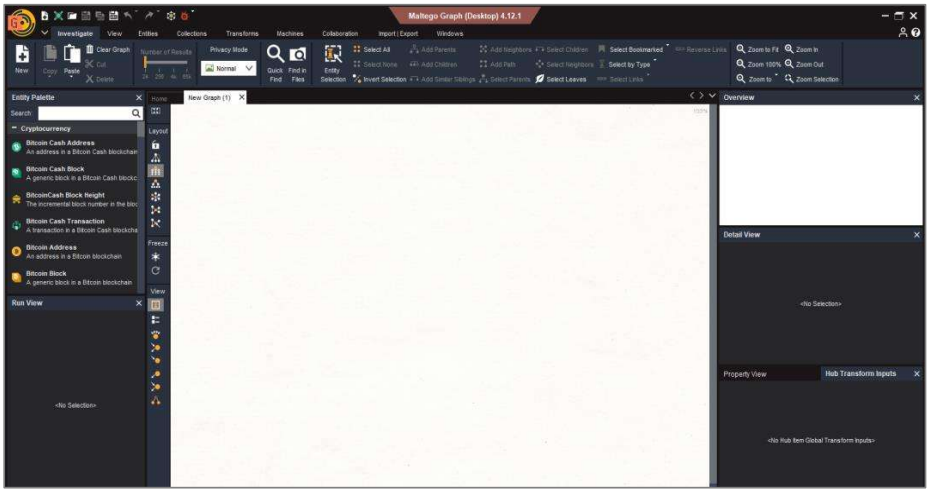


Fig. 4.20 New blank graph

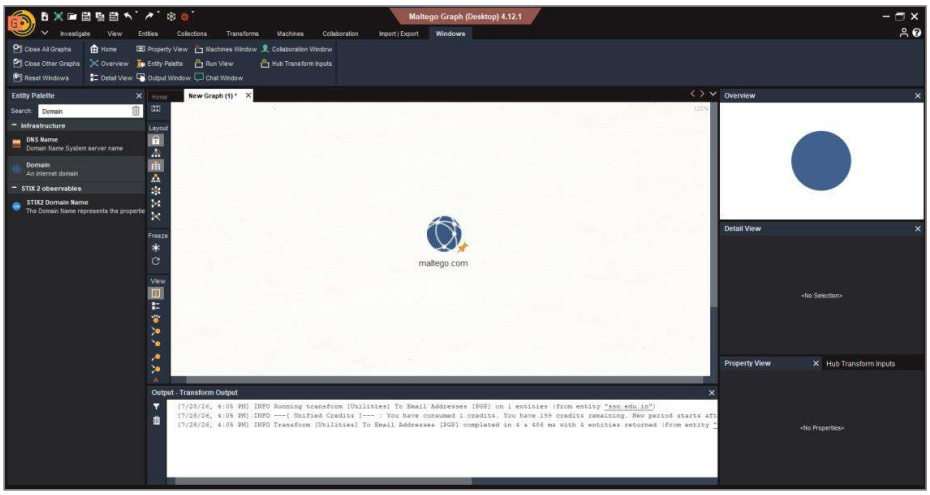


Fig. 4.21 Domain entity added to the graph

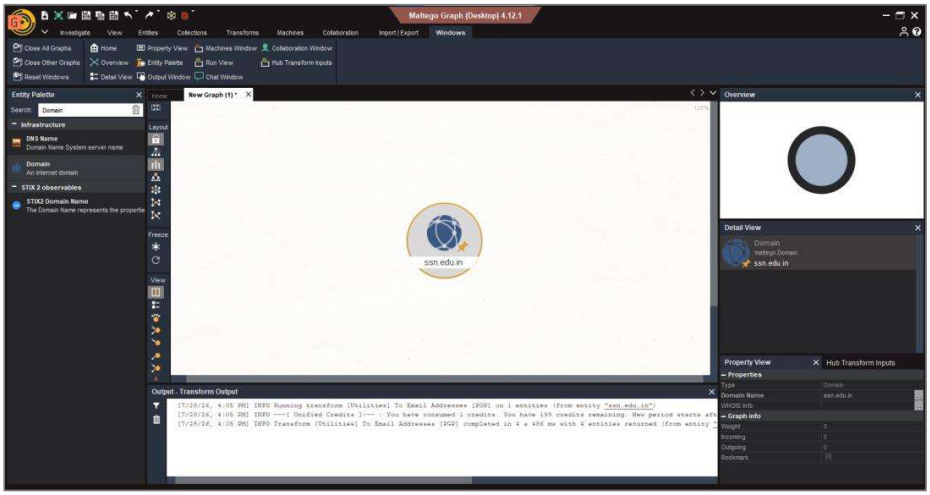


Fig. 4.22 Target domain ssn.edu.in set on the Domain entity

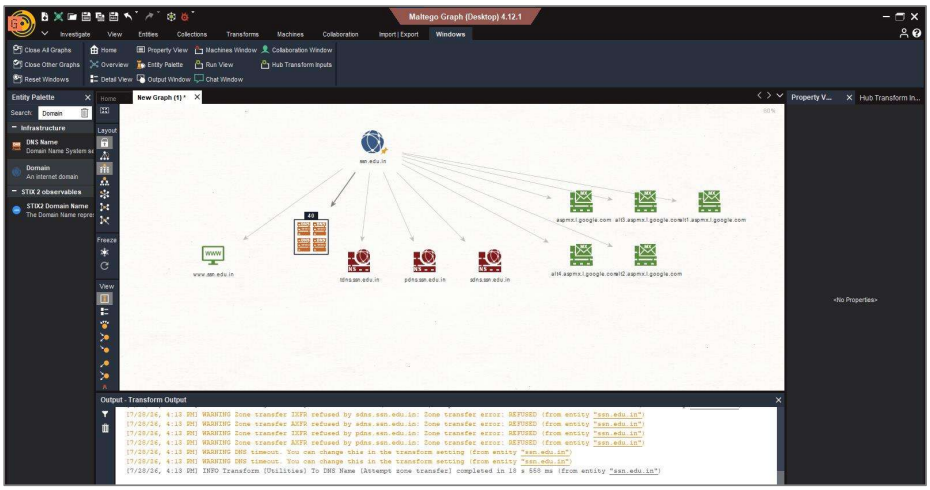


Fig. 4.23 DNS transform – name servers, web host and mail servers discovered

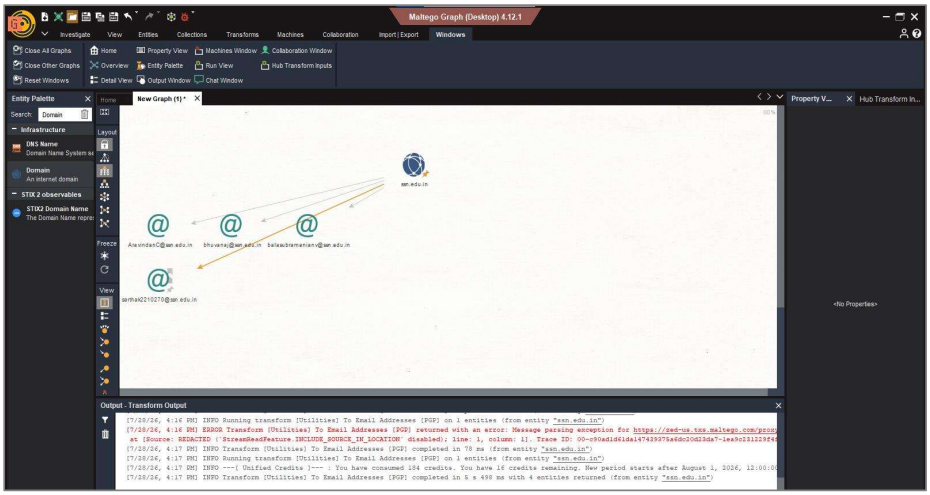


Fig. 4.24 E-mail transform – related e-mail addresses discovered

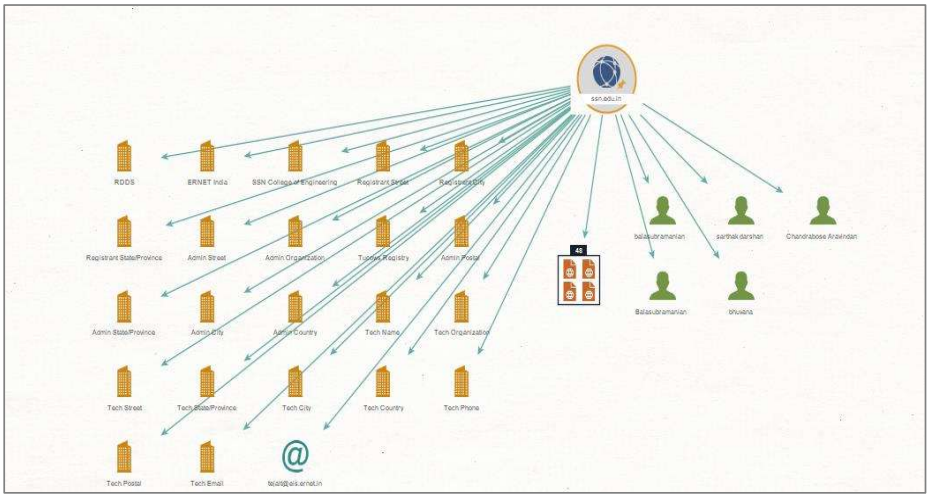


Fig. 4.25 Persons / WHOIS transform – registrant, admin and tech contact entities

4.2 Enriching Findings with Shodan

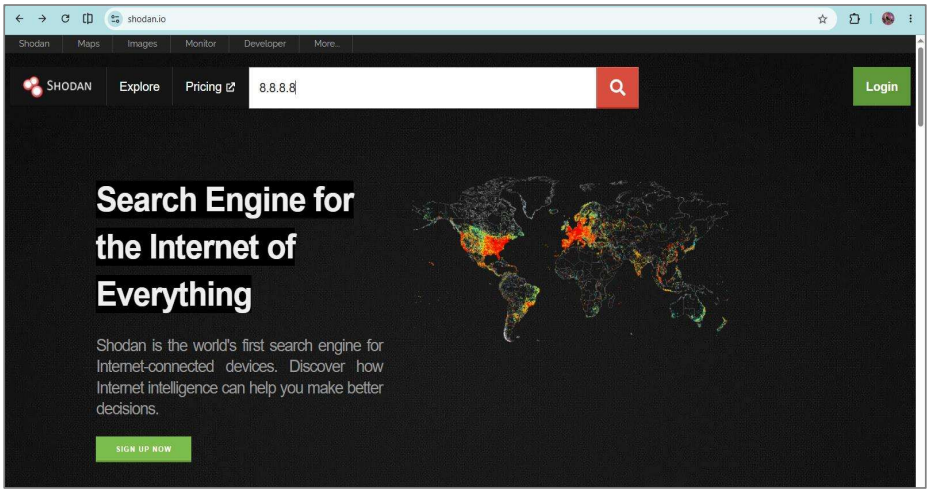


Fig. 4.26 Shodan search for the IP address 8.8.8.8

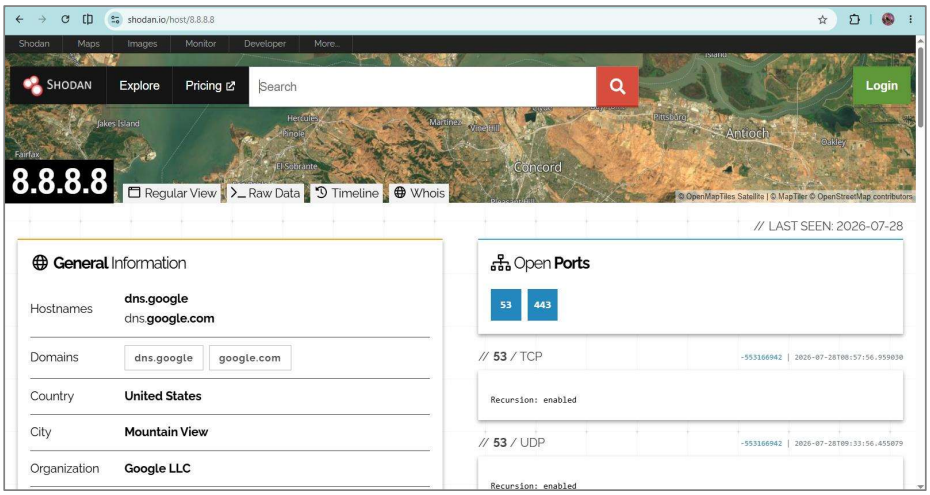


Fig. 4.27 Shodan host page for 8.8.8.8 – general information and open ports

Learning Outcomes:

- Registered for, installed and activated Maltego Community Edition and identified its main interface areas: entities, transforms, graph canvas and Machines.
- Built an entity graph from a single authorised domain using DNS, e-mail and persons/WHOIS transforms.
- Traced relationships between domains, name servers, mail servers, e-mail addresses and registration contacts and identified shared infrastructure.
- Used Shodan to enrich and cross-check an IP address from the graph with open ports, hostnames and location information.
- Understood that OSINT data is public but may be stale, so it must be validated, and that investigations must be limited to authorised targets.

Ex. No. 5 **Date:** 12/08/2026

View and capture network traffic using Wireshark

Objective:

To capture, inspect and control network and web traffic in the isolated lab and to observe how defensive tools react to it: analysing packets with Wireshark, intercepting HTTP/HTTPS requests with Burp Proxy, enforcing an access-control policy with the pfSense firewall, and detecting authorised test traffic with the Snort intrusion detection system.

Tools:

Wireshark, Burp Suite (Proxy), pfSense, Snort; Kali Linux with an authorised vulnerable target (DVWA / Metasploitable) on the lab network.

Methodology:

Traffic is first captured and analysed packet by packet, then web requests are intercepted through a proxy, firewall rules are applied to permit required services and block the rest, and authorised test traffic is generated to confirm that the IDS raises alerts. All activity is limited to the isolated lab network and to traffic the tester is permitted to capture. The experiment is carried out in four parts (7.1 to 7.4).

5.1 Packet Capture and Analysis with Wireshark

Principle: Wireshark decodes captured network frames and shows protocol fields in detail. Analysts can inspect addresses, ports, flags, headers and payload where available. The window has three panes: the packet list (one-line summary of each frame), the packet details (decoded protocol layers) and the packet bytes (raw hexadecimal data). Display filters isolate the relevant traffic from a large capture without deleting the rest.

1. Start Wireshark (as a user allowed to capture) and select the authorised lab interface, or open the supplied PCAP file with File > Open.
2. Start the capture, generate test traffic (ping the target, browse to the target web page, perform a DNS lookup) and stop the capture.
3. Review the packet list and the protocol distribution (Statistics > Protocol Hierarchy).
4. Apply display filters for the hosts and protocols of interest.
5. Select a packet and inspect its layers in the packet details pane and its bytes in the bytes pane; follow a TCP stream where required.
6. Record the important or suspicious packets by packet number and timestamp.

```
ip
tcp
udp
http
dns
```

icmp

```
tcp.port == 80
ip.addr == <ip>
```

Observation: The capture shown in the screenshots contains 32 frames: ICMP echo requests with matching replies, a DNS query and response for the name dvwa.lab, and two HTTP exchanges with their TCP three-way handshakes and closing FIN/ACK segments – a GET /login.php answered by 200 OK, and a form POST to /login.php answered by 302 Found. The filters icmp and http reduced the list to just those frames. Because HTTP is not encrypted, the request line, headers and form data can be read directly in the packet details and bytes panes, which shows why credentials must not be sent over plain HTTP. Display filters only change what is shown, not what was captured, and only authorised traffic may be captured.

5.2 HTTP/HTTPS Interception with Burp Proxy

Principle: Burp Proxy sits between the browser and the web server as an intermediary, so every request and response can be inspected and, if needed, held back before it is forwarded. It shows the method, URL, headers, cookies, parameters and body of each request. HTTP History records all proxied traffic, and Repeater can resend a selected request for instructor-approved checks.

1. Start Burp Suite and either use its built-in browser or set the browser proxy to 127.0.0.1:8080.
2. Open Proxy > Intercept and switch Intercept on, then browse to the authorised vulnerable application (for example DVWA).
3. Inspect the intercepted request: request line, Host, Cookie header, parameters and body (for a login form, the username and password fields).
4. Click Forward to send the request on, and review the entry in Proxy > HTTP History.
5. Use Repeater only for parameter checks approved by the instructor, and switch Intercept off when finished.

Observation: The Proxy is the interception layer, and HTTP History keeps a record of every proxied request and response for later review. For HTTPS, the browser must trust Burp's local CA certificate, otherwise it shows certificate warnings. Interception must be used only against the authorised application.

5.3 Firewall Configuration with pfSense

Principle: pfSense is a firewall and router platform that controls traffic between interfaces such as WAN and LAN. A rule states an action (Pass or Block), protocol, source, destination and port, and rules are evaluated from top to bottom on the interface where the traffic enters. A secure policy follows least privilege: allow only the services that are required and block everything else.

1. Start the pfSense virtual machine and assign the WAN and LAN interfaces as instructed for the lab.
2. Open the pfSense web interface from a machine on the management (LAN) network.
3. Go to Firewall > Rules and select the LAN tab.
4. Create Pass rules for the required services (for example TCP 80 and 443 and DNS on port 53) and Block rules for the prohibited traffic (for example ICMP echo and Telnet on TCP 23); disable the default allow-all rule so that unlisted traffic is denied.
5. Save, click Apply Changes, and test from a LAN host that the permitted services work and the blocked traffic fails.
6. Record the final rule set and the test results.

Observation: Rule order and the interface on which a rule is placed decide the behaviour: Pass allows the matching traffic and Block denies it, and the first matching rule wins. The management interface must stay reachable so that the administrator is not locked out. The rule set above is an example and should be matched to the rules actually configured in the lab.

5.4 Network Intrusion Detection with Snort

Principle: Snort inspects network packets against a set of rules and raises an alert when traffic matches a signature. A rule consists of an action, protocol, source and destination addresses and ports, and options such as a message (msg) and a signature identifier (sid). Alerts must be read in context, because normal traffic can sometimes match a signature, and rules are tuned to reduce false positives.

1. Verify that Snort is installed and note its version, and identify the monitoring interface.
2. Set the authorised lab network in the HOME_NET variable of the Snort configuration file.
3. Validate the configuration with snort -T -c <config-file>; Snort reports that the configuration was successfully validated.
4. Add the instructor-approved detection rules to the local rules file (an ICMP echo rule and an HTTP login-POST rule were used).
5. Run Snort in IDS mode on the lab interface with the alert mode enabled.
6. Generate the authorised test traffic (ping the target and send an HTTP POST to the login page) and read the alerts in the alert log.

```
snort -V
snort -T -c <config-file>
snort -A fast -q -c <config-file> -i <interface> -l <log-dir>
alert icmp any any -> $HOME_NET any (msg:"LAB ICMP echo request detected"; itype:8;
sid:1000001; rev:1;)
alert tcp any any -> $HOME_NET 80 (msg:"LAB HTTP login POST detected";
flow:to_server,established; content:"POST /login.php"; sid:1000002; rev:1;)
```

Observation: Snort 2.9.20 validated the configuration, and in IDS mode it raised one alert (sid 1000001) for each of the three ICMP echo requests and one alert (sid 1000002) for the HTTP POST to /login.php, each with its timestamp, message, protocol and source and destination. A network IDS only monitors and reports; it does not block traffic. Precise rules reduce false positives, and only the authorised lab interface and test traffic should be used.

Screenshots:

5.1 Packet Capture and Analysis with Wireshark

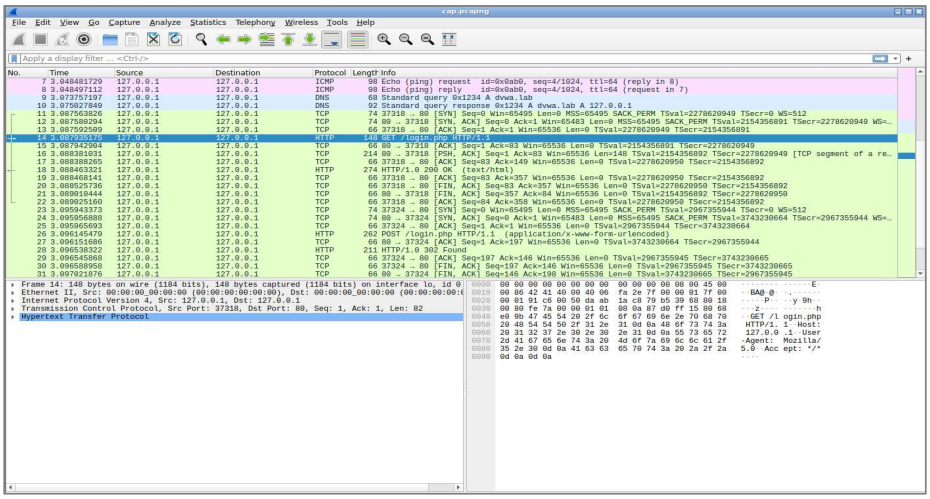


Fig. 5.1 Wireshark – captured packets with the HTTP GET request selected (packet list, details and bytes panes)

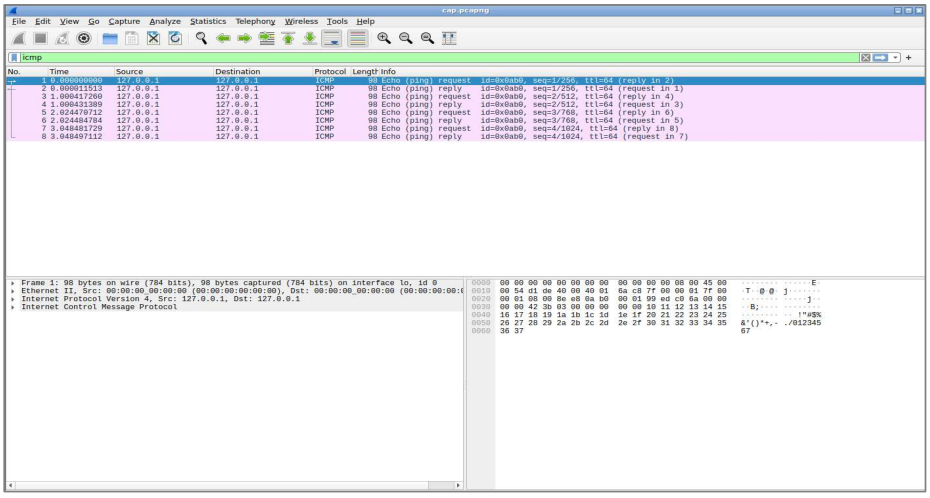


Fig. 5.2 Display filter icmp – ping echo requests and replies

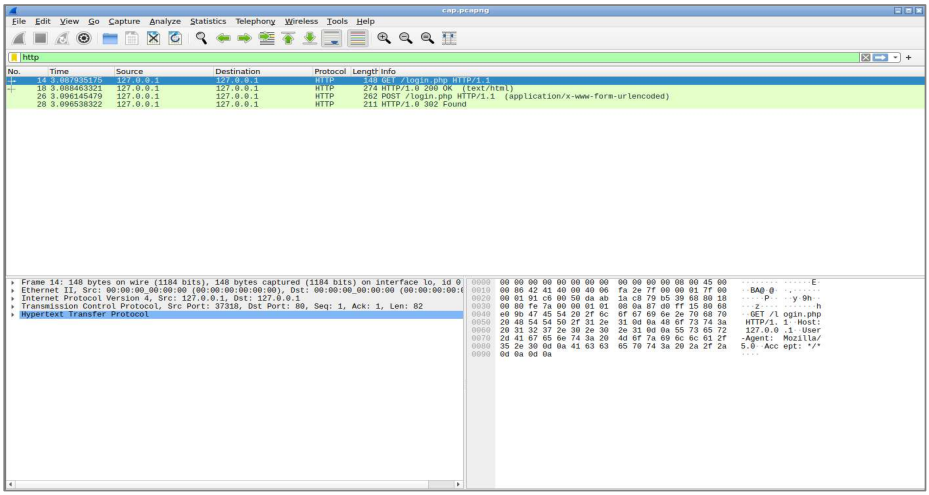


Fig. 5.3 Display filter http – GET and POST requests with their responses

5.2 Network Intrusion Detection with Snort

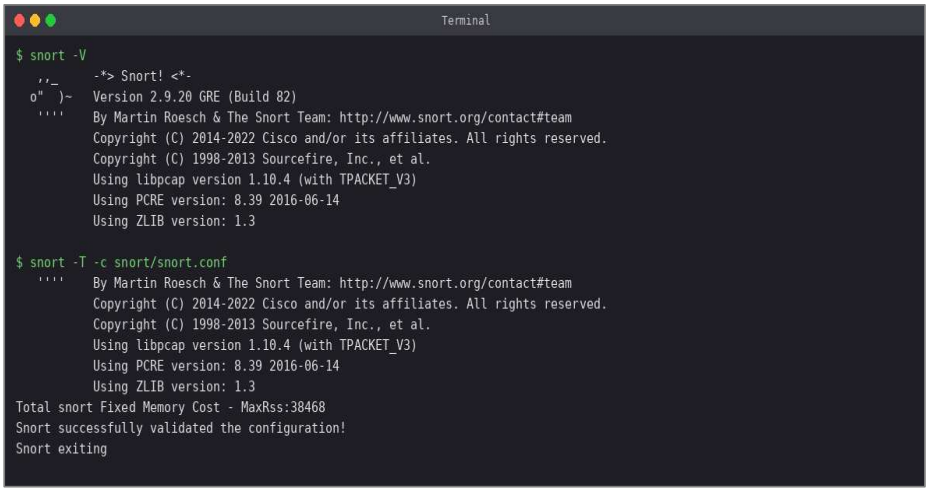


Fig. 5.4 Snort version and successful configuration validation (snort -V, snort -T -c)

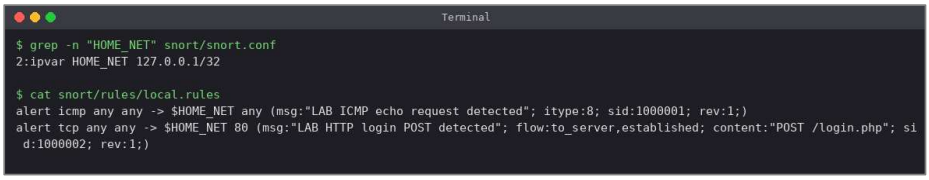


Fig. 5.5 HOME_NET setting and the local detection rules



Fig. 5.6 Snort IDS mode – alerts generated by the ping and HTTP POST test traffic

Learning Outcomes:

- Captured and analysed network traffic with Wireshark, using display filters to isolate ICMP, DNS, TCP and HTTP packets and inspecting their fields.
- Understood how an intercepting proxy such as Burp exposes web requests, cookies and parameters, and why HTTPS inspection needs a trusted certificate.
- Created Pass and Block rules in pfSense following the least-privilege principle and tested their effect.
- Configured Snort, wrote simple detection rules, validated the configuration and interpreted the alerts raised by test traffic.
- Understood that traffic capture, interception and testing must be limited to the authorised lab network.

Ex. No. 6 **Date:** 09/09/2026

Scan the target using tools like Nessus

Objective:

To assess an authorised lab target for known vulnerabilities using automated scanners: a web-server scan with Nikto and a network vulnerability scan with Nessus Essentials. The findings are to be reviewed by severity, CVE references and remediation guidance, treating scanner output as leads that must be validated before reporting.

Tools:

Nikto (Kali Linux); Nessus Essentials (Tenable); lab targets: Metasploitable / DVWA virtual machines; cve.org and nvd.nist.gov for CVE look-up.

Methodology:

Vulnerability assessment is the systematic process of identifying, classifying and prioritising security weaknesses. The experiment is carried out in two parts (6.1 and 6.2). Both scanners are active: they send requests to the target and are visible in its logs, so they are run only against systems that are owned or explicitly authorised for testing.

6.1 Web Server Vulnerability Assessment with Nikto

Principle: Nikto is a free, open-source command-line web-server scanner. It checks for thousands of known issues – outdated server software, dangerous or default files and scripts, insecure HTTP headers and common misconfigurations. A reported item is a lead, not automatic proof of exploitability, and scanners can produce false positives.

1. Install Nikto (usually already present on Kali) and verify the version.
2. Confirm the IP address of the designated lab web server with ip a or the VirtualBox network settings.
3. Run the scan against the lab target and let it finish; a summary and end time are printed.
4. Review the findings (each line starts with +), classify the important ones and note the recommended fixes.

```
sudo apt install nikto -y
nikto -Version
nikto -Help
nikto -h <lab-target-ip-or-url>
nikto -h <target> -p 443 -ssl -o report.html -Format html
```

Common options: -h target host (required); -p custom port (default 80); -ssl force HTTPS; -o output file; -Format txt/html/csv/xml; -Tuning restrict the scan to selected test categories.

Observations (from the example run in the screenshots, Nikto v2.5.0):

- The scan reported the target IP, host name, port 443 and the SSL certificate details (subject, cipher TLS_AES_256_GCM_SHA384, issuer Let's Encrypt).

- Server header disclosure (nginx); the anti-clickjacking X-Frame-Options header is not present; the X-Content-Type-Options header is not set.
- Many cookies were created without the secure and httponly flags; robots.txt contains 11 entries that should be reviewed manually; the host name does not match the certificate's name.
- Content-Encoding "deflate" was flagged as possibly exposing the server to the BREACH attack.
- The scan ended after the error limit (20) was reached, reporting 30 items in 440 seconds. Findings are leads and must be validated.

Recommendation: remove default content, hide version information, add the missing security headers, set secure/httponly flags on cookies and patch outdated components.

6.2 Vulnerability Assessment with Nessus

Principle: Nessus (Tenable) is an automated vulnerability scanner that compares the services and configurations it finds against a continuously updated database of checks (plugins). It reports missing patches, misconfigurations, weak credentials and outdated software, ranked by severity with CVE references and remediation guidance. Nessus Essentials is the free edition, limited to 16 IP addresses.

A. Install and activate Nessus Essentials

1. Download the Windows x64 .msi installer from tenable.com/downloads/nessus, run it, accept the licence agreement, keep the default folder and click Install.
2. Verify or start the service from Services (services.msc) or an administrator command prompt.
3. Register at tenable.com/products/nessus with an e-mail address to receive a free Essentials activation code.
4. Open https://localhost:8834, click Advanced and accept the self-signed certificate warning.
5. Select Nessus Essentials, enter the activation code, create an administrator user name and password, and wait 15–45 minutes while the plugin feed downloads and compiles.

```
net start "Tenable Nessus"
```

B. Create and run a scan

1. From My Scans click New Scan and choose a scan template (Basic Network Scan; the example run used the Advanced Scan template).
2. Enter a descriptive name and the target IP address of the authorised lab VM in the Targets field.
3. Click Save, then click the Play (Launch) icon next to the saved scan and monitor progress until completion.

C. Review severity and CVEs

1. Open the completed scan and review the Hosts and Vulnerabilities tabs.
2. Open a Critical or High item and read the description, solution, output and the See Also / References section to find the CVE-YYYY-NNNNN numbers.
3. Search the CVE on cve.org or nvd.nist.gov to inspect the CVSS vectors, impact metrics and patch recommendations.

Severity ratings:

- **Critical** – immediate, severe risk; complete system compromise, remote code execution or full administrative access.

- **High** – significant risk; unauthorised data access, privilege escalation or full service disruption.
- **Medium** – moderate risk; usually needs conditional access or specific configuration to exploit.
- **Low** – minor risk; misconfigurations, protocol weaknesses or informational disclosure.
- **Info** – non-vulnerable findings giving operational or contextual detail gathered during discovery.

Observations (from the example scan of a Metasploitable 3 target, 192.168.1.9):

- The scan completed against one host and the results were grouped by severity (Critical, High, Medium, Low, Info) in the Hosts view.
- A Critical finding, plugin 60085 "PHP 5.3.x < 5.3.15 Multiple Vulnerabilities", was reported on port 8585/tcp: the installed version was PHP 5.3.10 and the fixed version is 5.3.15. Its CVEs include CVE-2012-2688 and CVE-2012-3365.
- CVE-2012-3365 (published 2012-07-20) concerns the SQLite functionality in PHP before 5.3.15, which allows remote attackers to bypass the open_basedir protection via unspecified vectors.
- Solution: upgrade PHP to version 5.3.15 or later. CVSS gives a standard severity score, but severity must be read together with the asset context; important findings should be validated before reporting.

Screenshots:

6.1 Web Server Vulnerability Assessment with Nikto



Fig. 6.1 Installing / verifying Nikto



Fig. 6.2 nikto -Version



Fig. 6.3 Nikto scan started – target, port and SSL information



Fig. 6.4 Nikto findings (server, headers and cookie issues)

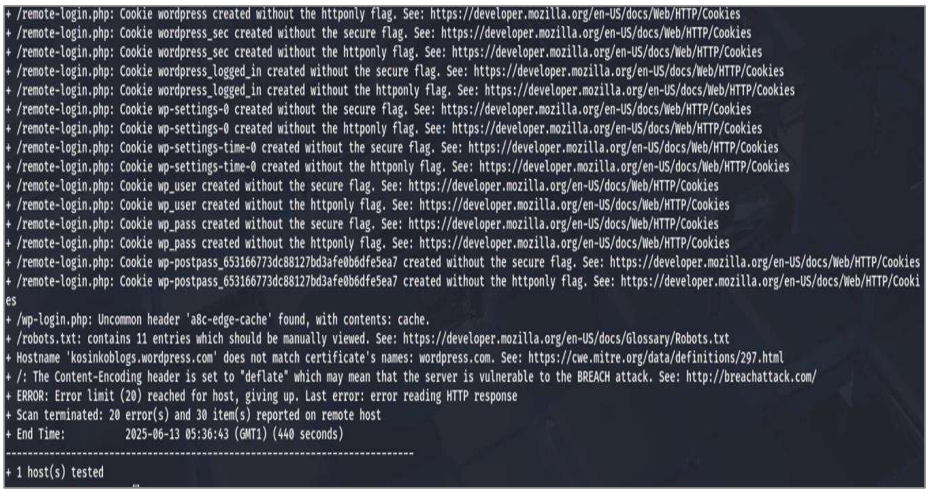


Fig. 6.5 Nikto scan summary and end time

6.2 Vulnerability Assessment with Nessus

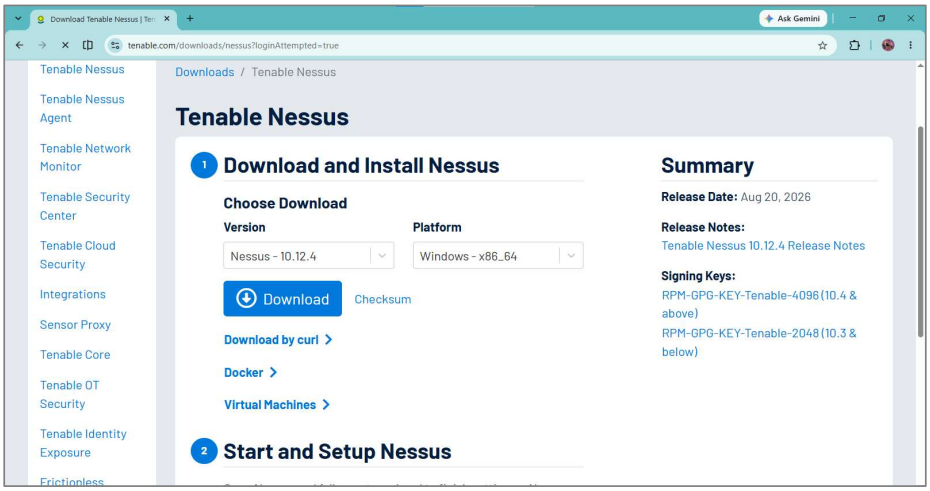


Fig. 6.6 Nessus download page

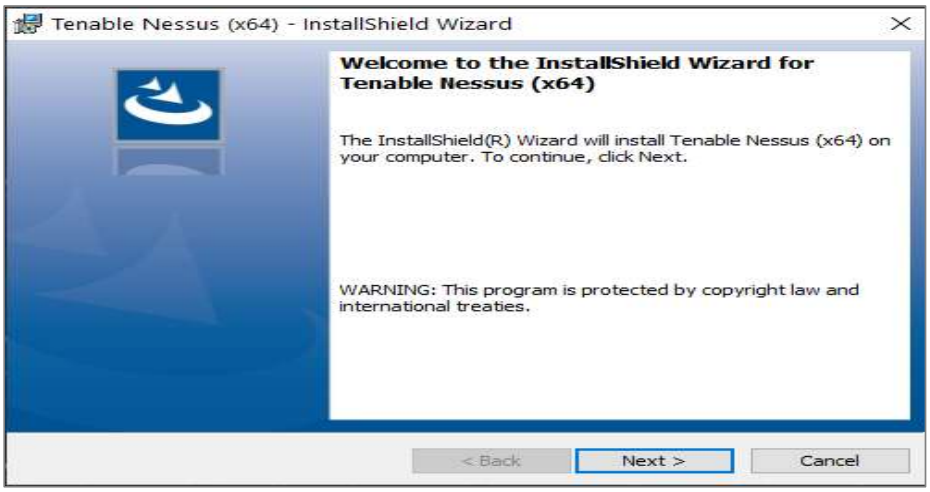


Fig. 6.7 Nessus installation wizard

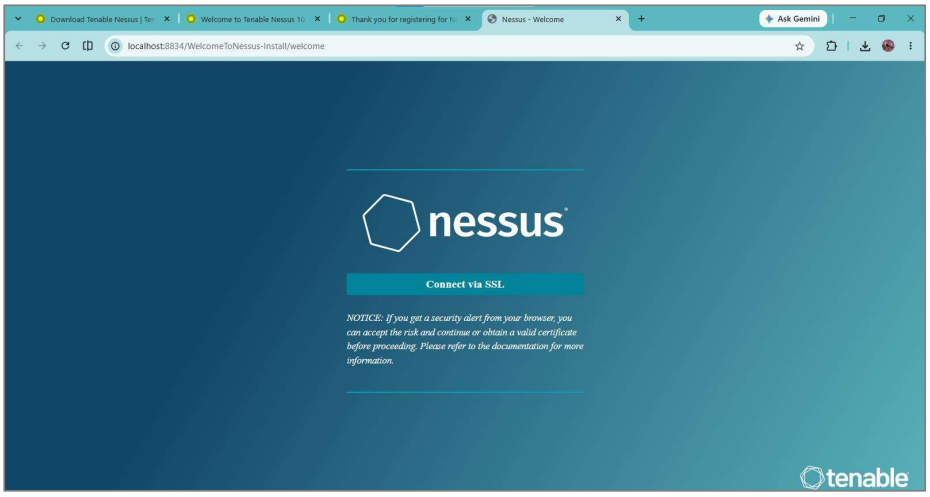


Fig. 6.8 Opening the Nessus web interface (https://localhost:8834)

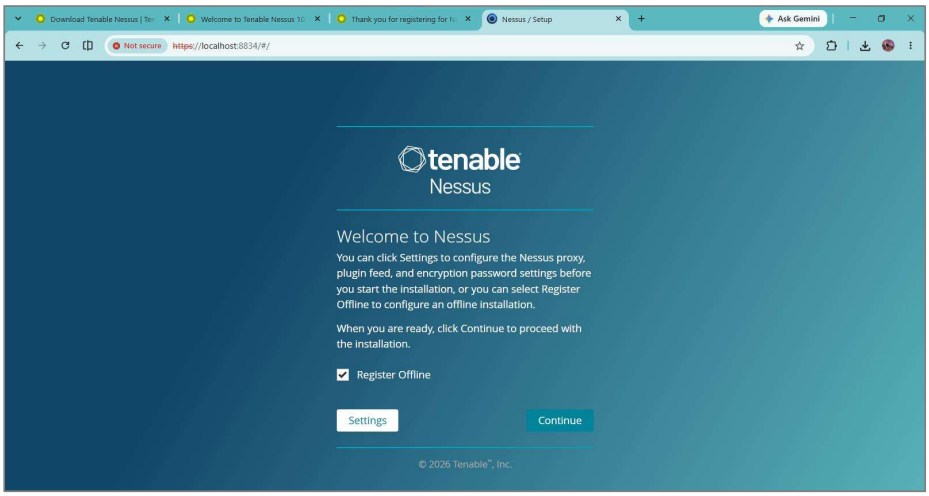


Fig. 6.9 Welcome to Nessus – setup

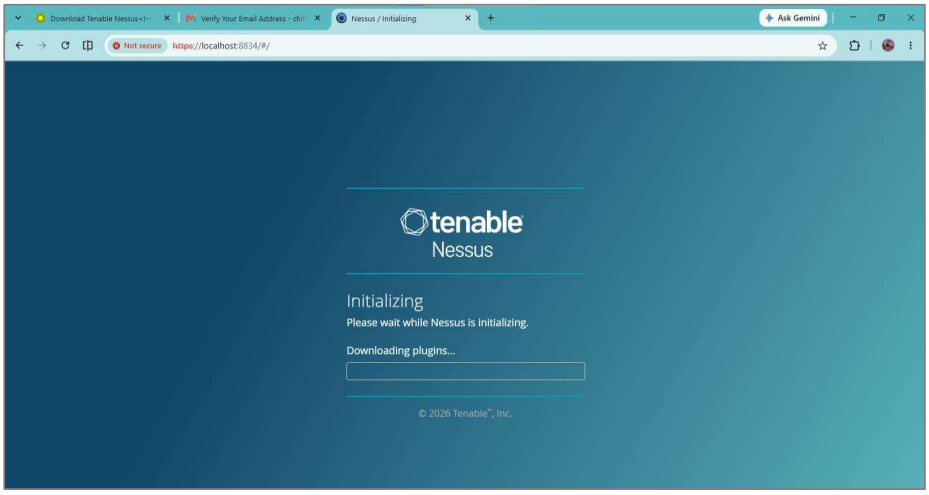


Fig. 6.10 Plugin feed download (initialising)

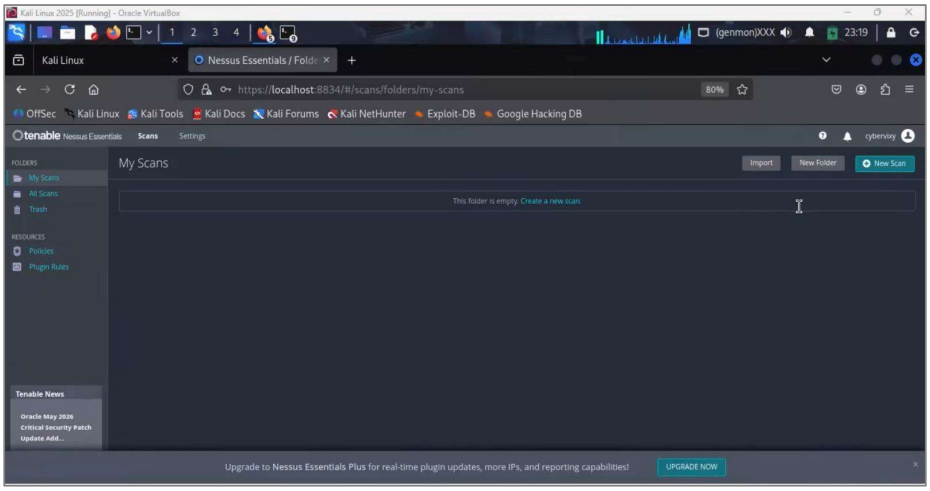


Fig. 6.11 My Scans page

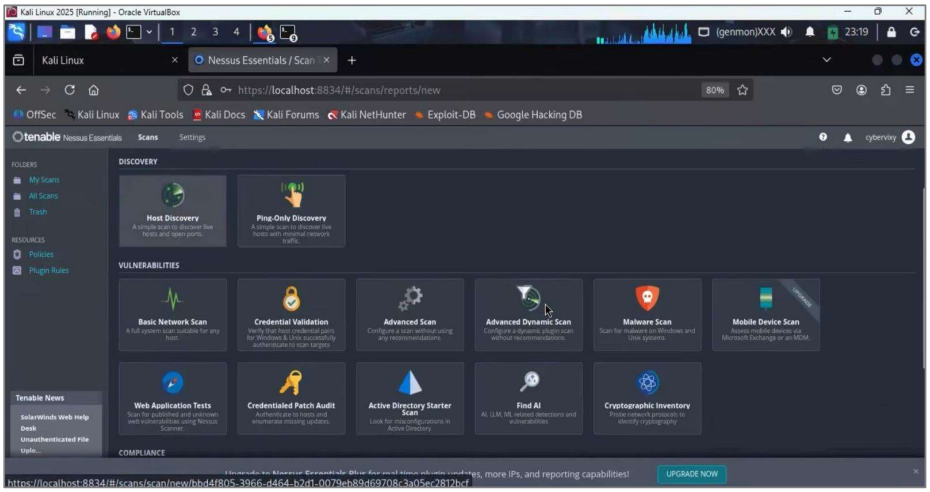


Fig. 6.12 Scan templates

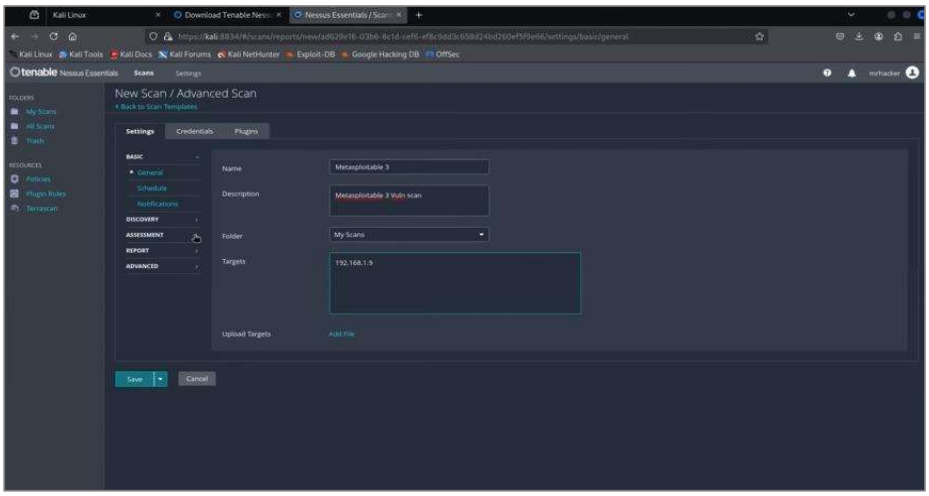


Fig. 6.13 New scan configuration – name and target

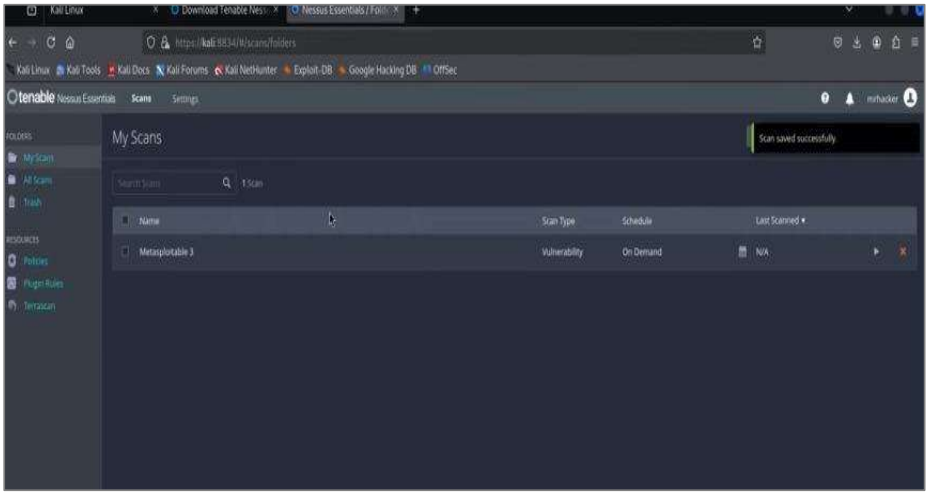


Fig. 6.14 Scan saved in My Scans

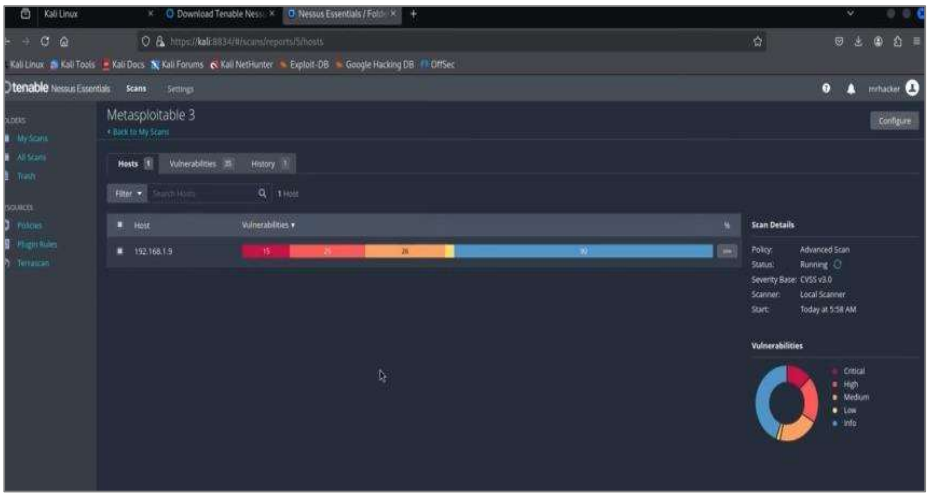


Fig. 6.15 Scan results – host with severity summary

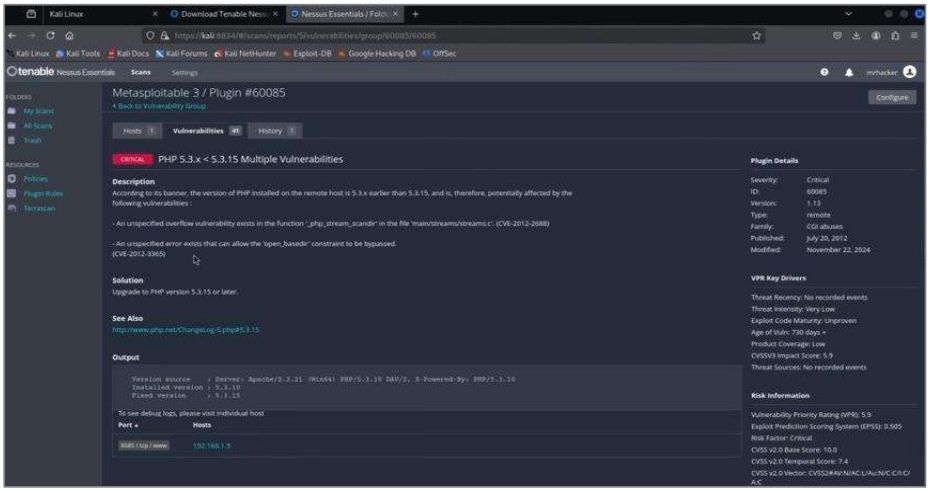


Fig. 6.16 Critical finding: PHP 5.3.x < 5.3.15 Multiple Vulnerabilities (plugin 60085)

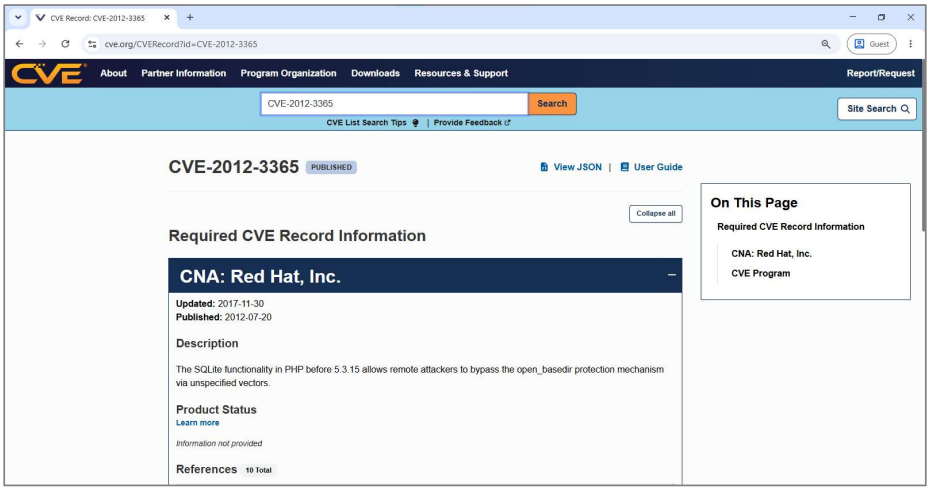


Fig. 6.17 CVE-2012-3365 record on cve.org

Learning Outcomes:

- Scanned an authorised web server with Nikto and interpreted its output: server disclosure, missing security headers, insecure cookies and default files.
- Installed and activated Nessus Essentials, configured a scan of an authorised lab target and generated a vulnerability report.
- Classified findings by severity (Critical, High, Medium, Low, Info) and traced them to CVE references and CVSS scores.
- Distinguished scanner output from validated findings and prioritised remediation using severity and context.

Ex. No. 7 **Date:** 16/09/2026

Information gathering using tools like Robtex

Objective:

To perform passive footprinting of an authorised domain using Robtex, a free web-based network and DNS intelligence tool: map its DNS records, discover related domains and infrastructure, review historical DNS data, identify network/ASN ownership information, and cross-check the important findings against other public sources.

Tools:

Robtex (robtex.com) in a web browser; WHOIS, crt.sh and DNSdumpster for cross-checking. No account or installation is required.

Methodology:

7.1 Network Intelligence with Robtex

Principle: Robtex aggregates publicly available DNS, routing and network-registration data into one browsable interface. It correlates domains, IP addresses, name servers, mail servers and hosting infrastructure and presents them as lists and an interactive graph, so relationships appear that a plain WHOIS or nslookup query would not show. It is purely passive: only Robtex's own aggregated data is queried and nothing is sent to the target's infrastructure.

1. Open https://www.robtex.com in any web browser.
2. Type the authorised training domain (ssn.edu.in) into the search bar and press Enter.
3. Review the summary panel: resolved IP address(es), hosting details, name servers (NS), mail servers (MX), TXT and SOA records, and the DNSSEC status.
4. Scroll to the Subdomains and Same first word lists to see related names and neighbouring domains.
5. Open DNS History and DNS Trace to compare former and current records, and check the delegation chain and authoritative response.
6. Look up a resolved IP address to identify the network range and ASN (Autonomous System Number) it belongs to.
7. Cross-reference the key findings with WHOIS, crt.sh and DNSdumpster before accepting them.

Interpreting DNS records:

- **A** – maps the domain to an IPv4 address.
- **MX** – identifies the mail server(s) responsible for the domain's e-mail.
- **NS** – identifies the domain's authoritative name servers.

- **TXT** – often holds SPF/DKIM policies or domain-verification strings.

Observations:

- ssn.edu.in is served by three name servers (tdns, sdns and pdns.ssn.edu.in) whose A records point to addresses announced by Vodafone Idea and Bharti Airtel.
- Mail is handled by Google: the MX records are aspmx.l.google.com and alt1–alt4.aspmx.l.google.com, and the SPF (TXT) record includes _spf.google.com.
- DNS Trace reports the zone as Insecure (no DNSSEC; no DS record), with a total query time of 482 ms. Record counts: NS 3, MX 5, TXT 7, SOA 1.
- DNS History lists 12 records (8 active, 4 former), observed from 2015-05-05; the former MX hosts aspmx2–aspmx5.l.google.com were last seen on 2018-10-24, so historical data does not describe the current setup.
- The subdomain list includes names such as ssnweb, webs, lms, sase, pdns and www. The parent zone edu.in shows warnings such as "Not in parent delegation" for some name servers.
- Domains in the Same first word list (e.g. ssn.it, ssn.ch, ssn.xyz) only share a name; sharing an IP address, name or hosting provider does not prove common ownership, and reputation data is not proof of malicious activity.
- Because aggregated and historical data can be stale, important findings were cross-checked with WHOIS, crt.sh and DNSdumpster.

Screenshots:

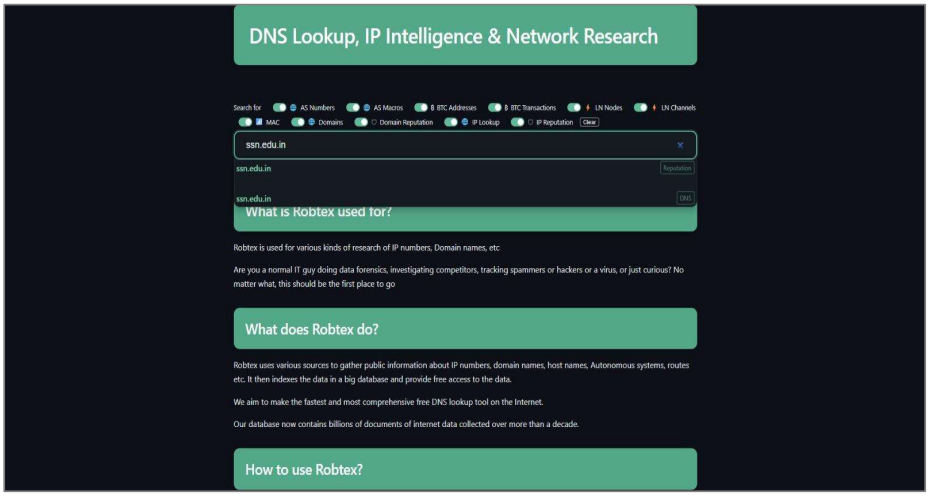


Fig. 7.1 Robtex home page with the domain entered in the search bar

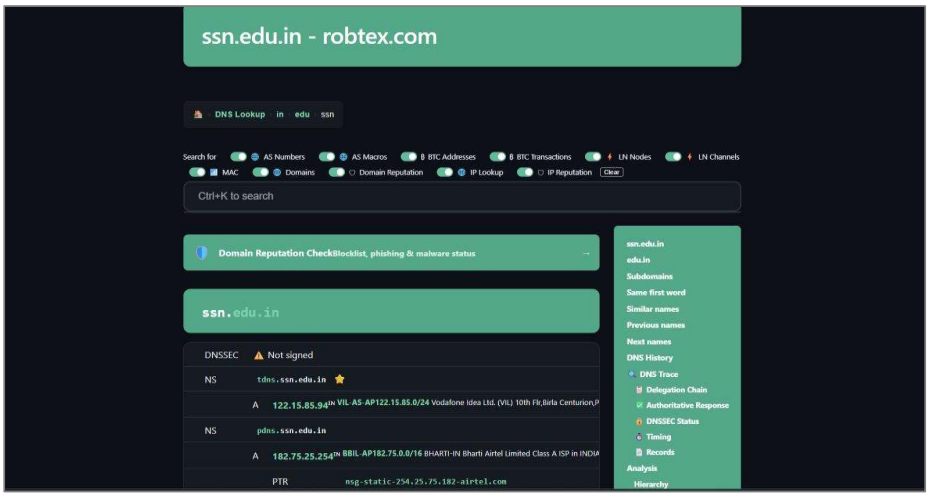


Fig. 7.2 Domain lookup result for ssn.edu.in – NS and A records

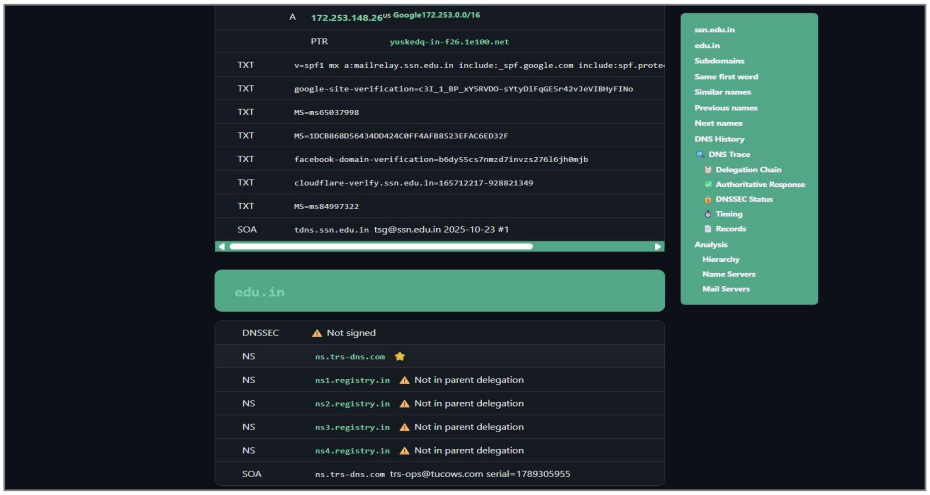


Fig. 7.3 Summary panel – TXT and SOA records and the parent zone edu.in

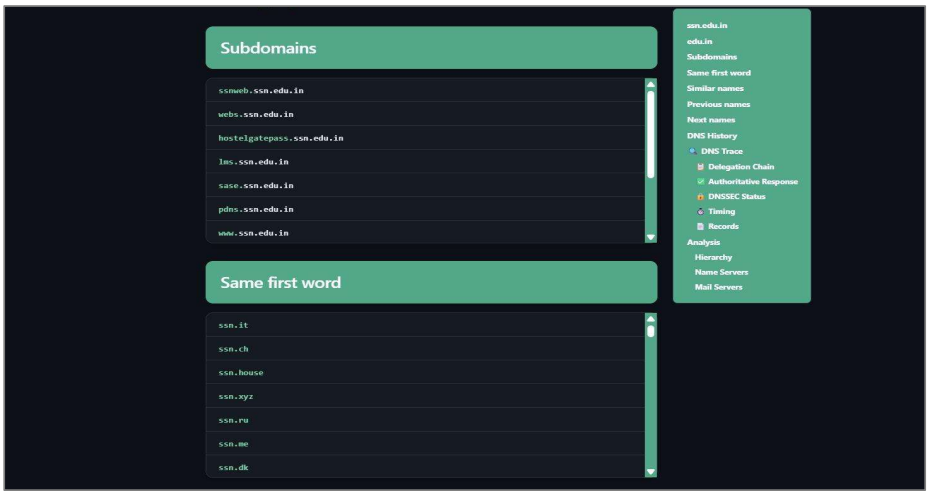


Fig. 7.4 Subdomains and Same first word lists

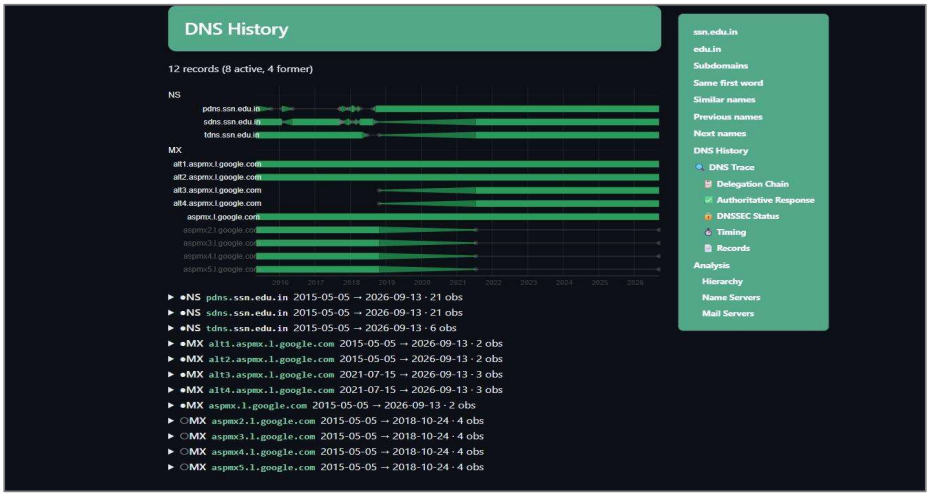


Fig. 7.5 DNS History – NS and MX records over time

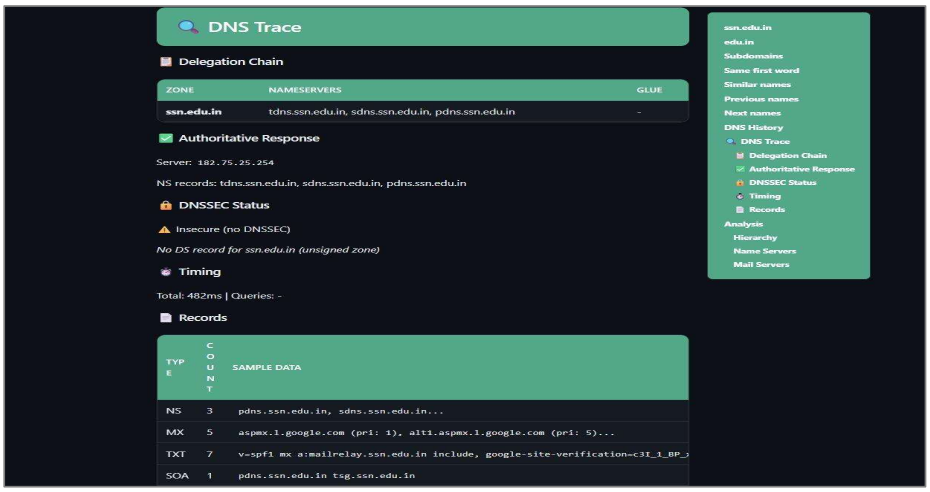


Fig. 7.6 DNS Trace – delegation chain, authoritative response, DNSSEC status and record counts

Learning Outcomes:

- Explained what Robtex is and how it complements other passive footprinting tools such as WHOIS, crt.sh and DNSdumpster.
- Performed a domain lookup and interpreted the returned A, MX, NS, TXT and SOA records.
- Used the subdomain, history and DNS Trace views to identify hosting, mail and name-server relationships and to separate historical from current data.
- Interpreted public network-intelligence relationships without confusing correlation with ownership or compromise, and cross-checked findings against other OSINT sources.

Ex. No. 8 **Date:** 23/09/2026

Automate discovery of vulnerabilities and match exploits using Armitage

Objective:

To use Armitage, a graphical front end for the Metasploit Framework, to organise authorised lab targets into a workspace, discover their services, automatically match applicable exploit modules with Find Attacks, and manage the resulting Meterpreter session, in an isolated lab network.

Tools:

Armitage, Metasploit Framework, PostgreSQL and Java (bundled on Kali Linux), with Nmap integration; authorised lab targets such as Metasploitable2/3 or another deliberately vulnerable image on an isolated host-only network.

Methodology:

Armitage is installed with its Metasploit and PostgreSQL dependencies, connected to the Metasploit RPC service, and a dedicated workspace is created. Lab hosts are discovered with Armitage's built-in Nmap scan, Find Attacks matches the recorded services against Metasploit's exploit database, and only the instructor-approved module is then launched against the authorised target so that the resulting Meterpreter session can be managed and documented.

Principle:

Armitage does not run exploits itself: it is a Java client that talks to Metasploit's RPC daemon (msfrpcd), which drives the Framework's exploit, payload and auxiliary modules and stores host, service and session data in a PostgreSQL database (Armitage GUI -> Metasploit RPC -> Framework -> PostgreSQL). This separation lets several Armitage clients share one Metasploit instance. Workspaces keep the hosts, scans and sessions of one exercise separate from any other sharing the same database. Find Attacks compares each host's recorded services with Metasploit's module list and tags the host with the exploits that could apply, while Hail Mary tries every applicable exploit automatically and is only suitable for a quick, noisy demonstration. A successfully exploited host turns red with a lightning-bolt icon and opens a Meterpreter session tab, from which sysinfo, getuid, file browsing, screenshots, privilege escalation and pivoting can all be run from the GUI.

Procedure:

1. Update the package index and install Armitage, which pulls in the Metasploit Framework and a compatible Java runtime as dependencies: sudo apt update && sudo apt full-upgrade -y, then sudo apt install armitage.
2. Start PostgreSQL and initialise the Metasploit database (once per machine): sudo service postgresql start, then sudo msfdb init; confirm with sudo msfdb status.
3. Verify the installation with armitage --help and msfconsole -v.

4. Launch Armitage with root privileges (sudo armitage); accept the default RPC host, port and username (127.0.0.1:55553, user msf) and allow Armitage to start msfrpcd automatically. The first connection can take a minute or two while Metasploit builds its module cache.
5. Create a new, empty workspace for this exercise (Workspaces > Manage) and switch to it before adding any hosts, so its hosts, scans and sessions stay separate from any other exercise.
6. Add the authorised lab hosts, either manually (Hosts > Add Hosts, one address per line) or by running Hosts > Nmap Scan > Quick Scan (OS detect) against the isolated lab network range; discovered hosts appear as icons in the Targets panel, labelled with their IP address and operating system where identifiable.
7. Select the discovered hosts and run Attacks > Find Attacks so Armitage compares their recorded services against its exploit database and tags each host with a matching Attack submenu; this can take a while the first time, since it has to load Metasploit's full module list.
8. Right-click the target, choose Attack, pick the instructor-approved module from the suggested list, review its options (RHOSTS, RPORT, LHOST/LPORT, Target) and click Launch.
9. Once the host turns red with a lightning-bolt icon, right-click it and choose Meterpreter > Interact to open a session tab; run basic commands (sysinfo, getuid) to confirm access, and document the attack path and remediation instead of going further.

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install armitage
sudo service postgresql start
sudo msfdb init
sudo msfdb status
armitage --help
msfconsole -v
sudo armitage
```

Observation:

Armitage installed cleanly together with metasploit-framework and openjdk as dependencies, and msfdb init created the msf and msf_test databases and confirmed the service was running. After connecting to 127.0.0.1:55553, the Targets panel listed the lab hosts (Windows 7, Ubuntu 20.04 and Metasploitable2 in the manual's example) as host icons. Hosts > Add Hosts and the Nmap Quick Scan both populated the same workspace with the authorised addresses, and Find Attacks tagged the Metasploitable2 host with matching modules, including exploit/unix/ftp/vsftpd_234_backdoor and exploit/multi/samba/usermap_script, with their RHOSTS, RPORT, LHOST and LPORT options shown before Launch. Running the usermap_script exploit opened a Meterpreter session, and sysinfo, getuid and screenshot were run from the session tab as point-and-click actions. Workspaces kept this exercise's hosts and session separate from the default workspace, and only the authorised Metasploitable2 lab target was attacked; Armitage and Metasploit must never be pointed at systems outside the isolated lab environment.

Screenshots:

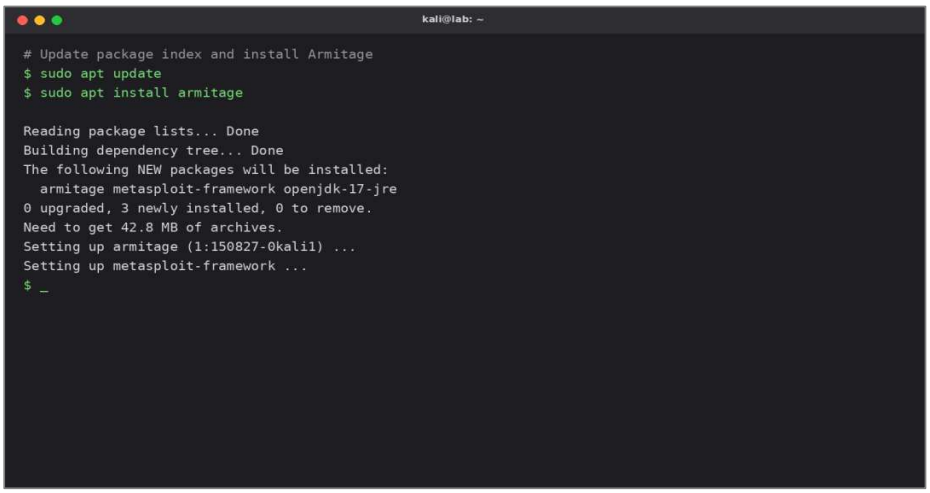


Fig. 8.1 Updating the package index and installing Armitage (pulls in metasploit-framework and openjdk)

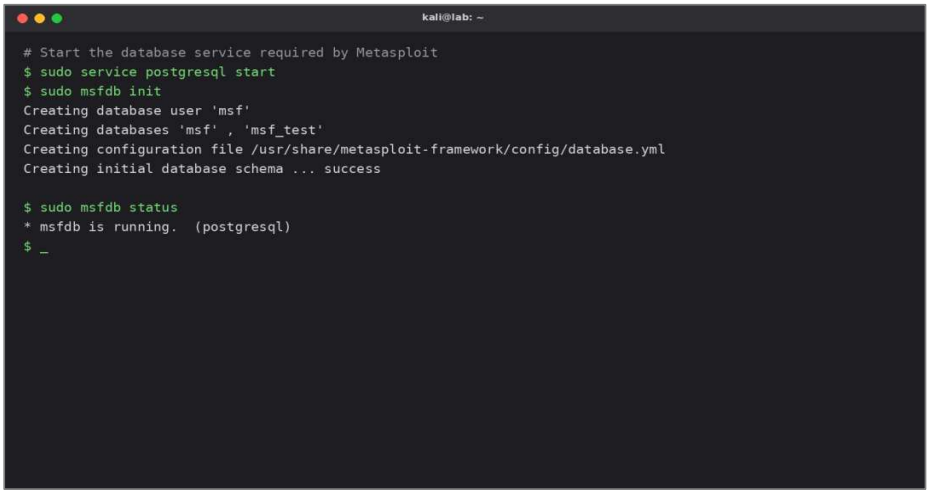


Fig. 8.2 Starting PostgreSQL and initialising the Metasploit database (msfdb init, msfdb status)

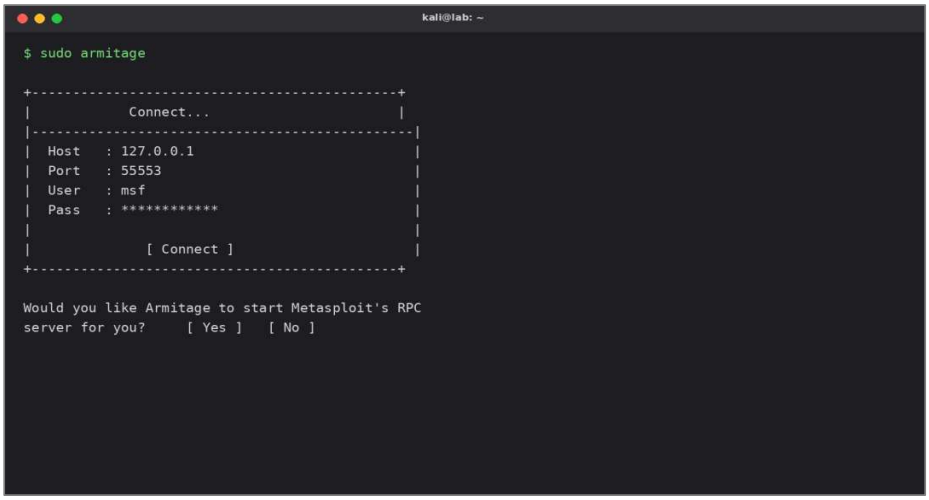


Fig. 8.3 Connecting Armitage to the Metasploit RPC service (127.0.0.1:55553) and starting msfrpcd

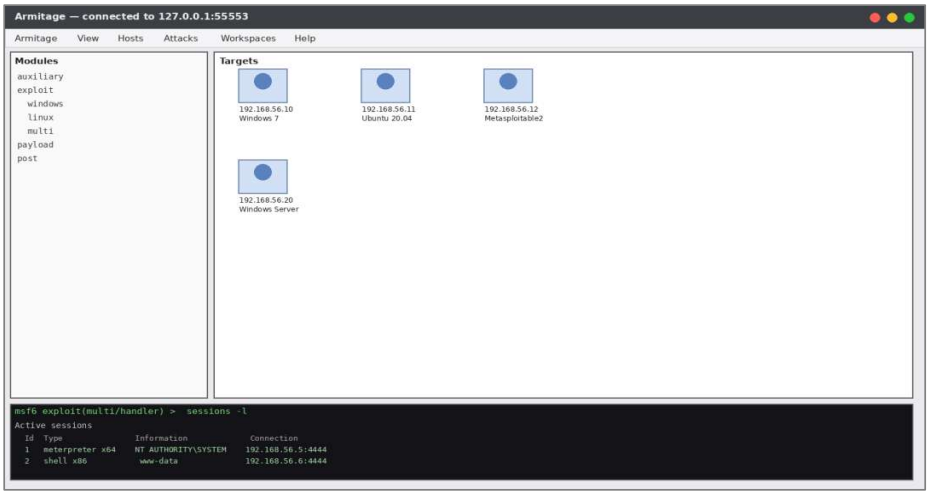


Fig. 8.4 Armitage main window connected to Metasploit, with the Modules panel and the Targets panel showing the lab hosts

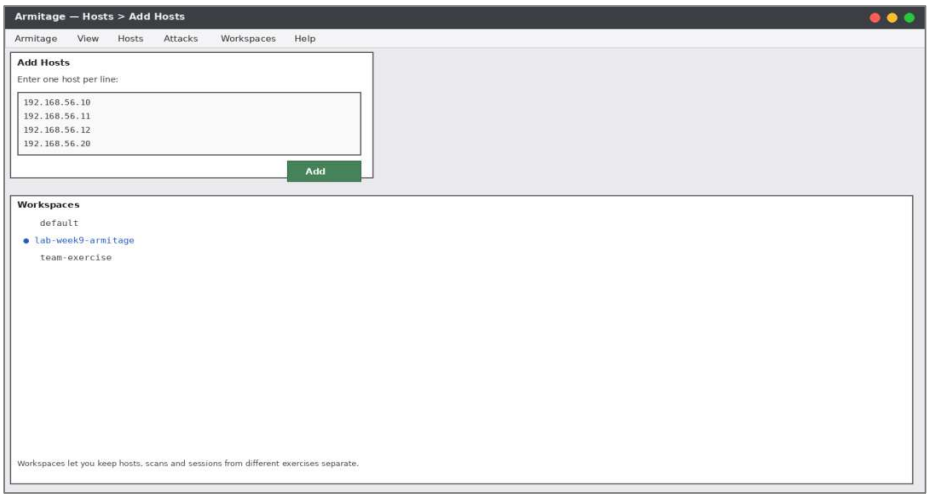


Fig. 8.5 Hosts > Add Hosts – authorised lab addresses added to the workspace

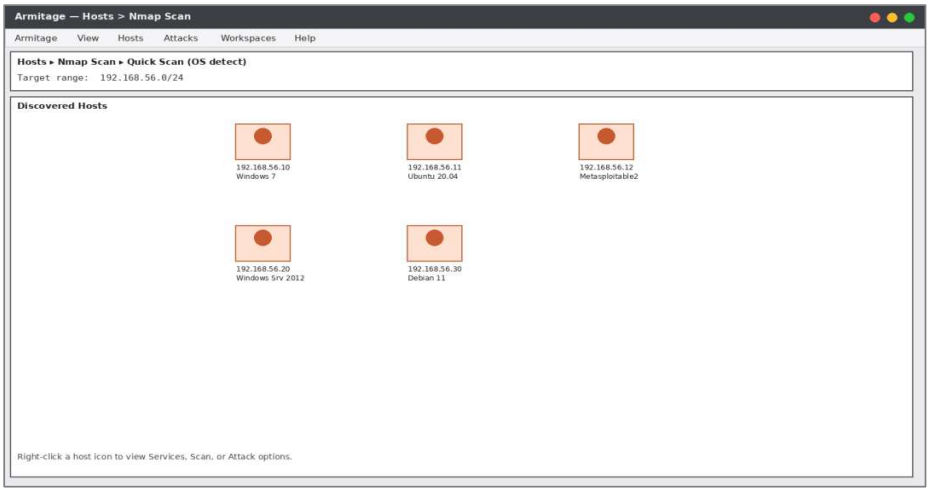


Fig. 8.6 Hosts > Nmap Scan > Quick Scan (OS detect) – discovered hosts in the Targets panel

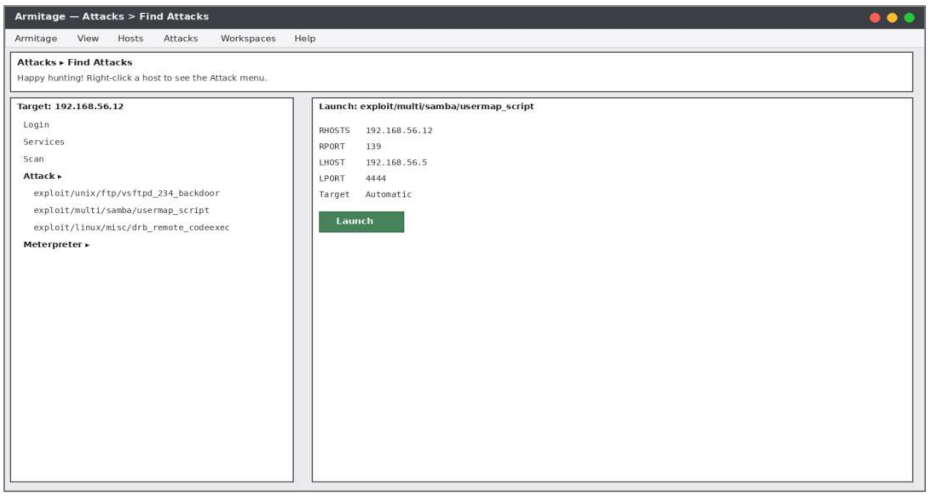


Fig. 8.7 Attacks > Find Attacks – matching exploit modules listed for the target, with module options before Launch

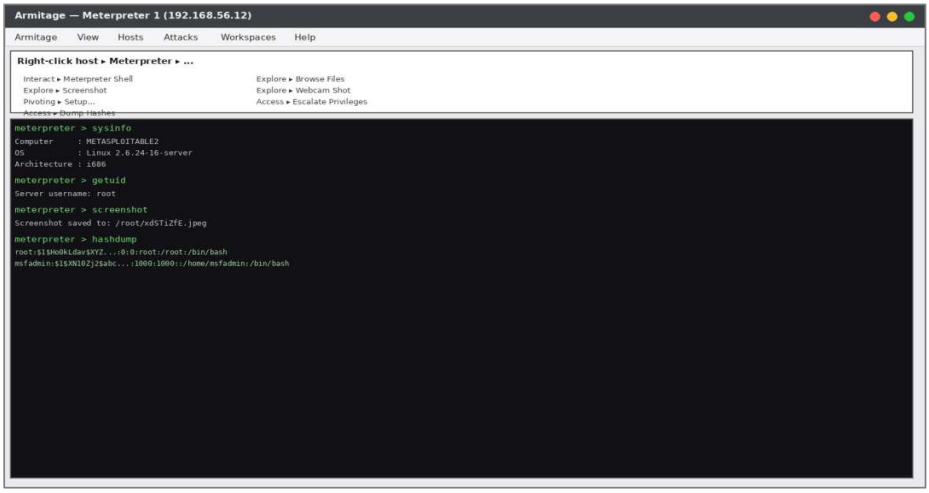


Fig. 8.8 Meterpreter session opened on the exploited host (sysinfo, getuid and further session commands)

Learning Outcomes:

Understood Armitage's architecture as a GUI front end that drives the Metasploit Framework through RPC and a shared PostgreSQL data store.

- Installed Armitage with its Metasploit, Java and PostgreSQL dependencies and connected it to the local RPC service.
- Created a workspace and added or discovered authorised lab hosts, keeping this exercise's data separate from other exercises.
- Used Find Attacks to match recorded host services against Metasploit's exploit database and launched only the approved module.